



UNIVERSIDAD DE CÓRDOBA
INSTITUTO DE ESTUDIOS DE POSTGRADO

**Máster Universitario en Inteligencia
Computacional e Internet de las Cosas**

Trabajo Fin de Máster

**Análisis y desarrollo de sistemas RAG
sobre SLM locales para la optimización
del acceso a la información de espacios
naturales e itinerarios de la Junta de
Andalucía**

Autor:

Anastasia Kuzmina

Directores:

Carlos García Martínez

Isabel Durán Muñoz

Título: Análisis y desarrollo de sistemas RAG sobre SLM locales para la optimización del acceso a información de espacios naturales de la Junta de Andalucía

Autor: Anastasia Kuzmina

Directores: Carlos García Martínez, Isabel Durán Muñoz

Resumen

Los asistentes conversacionales basados en inteligencia artificial generativa se han extendido a numerosos sectores como vía de acceso a la información. Sin embargo, los modelos generativos sobre los que se construyen presentan el problema de las alucinaciones, esto es, la generación de respuestas creíbles pero falsas. La arquitectura RAG (*Retrieval-Augmented Generation*) aborda esta limitación apoyando la generación en fragmentos recuperados de una base documental, y los modelos de lenguaje pequeños (*Small Language Models*, SLM) de ejecución local permiten controlar los datos y eliminar las dependencias de servicios externos. En este trabajo se diseña, implementa y evalúa un sistema conversacional basado en RAG sobre SLMs locales orientado a optimizar el acceso a la información de la plataforma de turismo natural de la Junta de Andalucía, en el marco del proyecto NATURCOR-IA. El sistema se construye a partir de un *pipeline* de ingesta de PDFs y páginas web, segmentación, generación de *embeddings* e indexación vectorial en FAISS, conectado a una cadena RAG sobre llama3.1:8b. Sobre este sistema se evalúan once configuraciones experimentales, frente al sistema de referencia, que cubren el tamaño de *chunk*, un *retriever* híbrido, *Multi-Query*, *Rewrite-Retrieve-Read* e inyección de metadatos. La evaluación se realiza con el *framework* RAGAS sobre tres métricas sin respuestas de referencia (*Faithfulness*, *Response Relevancy* y *Context Precision*) y se valida con un protocolo no paramétrico con corrección de Holm y tamaños del efecto. Los resultados muestran que únicamente el tamaño de *chunk* produce un efecto significativo frente al sistema de referencia, y que la configuración con *chunk* de 500 caracteres mejora la precisión del contexto recuperado con un efecto grande. Las técnicas más elaboradas no aportan mejoras medibles sobre este corpus y modelo, lo que constituye en sí mismo un resultado informativo para el caso de uso del proyecto.

Palabras clave: RAG, modelos de lenguaje pequeños, búsqueda semántica, bases de datos vectoriales, evaluación RAGAS, espacios naturales

Title: Analysis and development of RAG systems on local SLMs for optimizing access to information on natural spaces of the Junta de Andalucía

Author: Anastasia Kuzmina

Tutors: Carlos García Martínez, Isabel Durán Muñoz

Abstract

Conversational assistants based on generative artificial intelligence have spread across many sectors as a channel for accessing information. However, the generative models on which they are built suffer from hallucinations, that is, the generation of plausible but false responses. The RAG (*Retrieval-Augmented Generation*) architecture addresses this limitation by grounding generation in fragments retrieved from a document base, and Small Language Models (SLM) running locally add control over the data and remove dependencies on external services. This work designs, implements and evaluates a conversational system based on RAG over local SLMs aimed at optimizing access to the information on the natural tourism platform of the Junta de Andalucía, within the framework of the NATURCOR-IA project. The system is built from an ingestion *pipeline* for PDFs and web pages, with segmentation, *embedding* generation and vector indexing in FAISS, connected to a RAG chain on top of llama3.1:8b. Eleven experimental configurations are evaluated on this system, against the reference system, covering *chunk size*, a hybrid *retriever*, *Multi-Query*, *Rewrite-Retrieve-Read* and *metadata* injection. The evaluation is carried out with the RAGAS *framework* on three reference-free metrics (*Faithfulness*, *Response Relevancy* and *Context Precision*) and is validated through a non-parametric protocol with Holm correction and effect sizes. The results show that only *chunk size* produces a significant effect against the reference system, and that the configuration with a *chunk size* of 500 characters improves the precision of the retrieved context with a large effect. The more elaborate techniques do not yield measurable improvements on this corpus and model, which is itself an informative outcome for the use case of the project.

Keywords: RAG, small language models, semantic search, vector databases, RAGAS evaluation, natural spaces

Índice general

Resumen	i
Abstract	ii
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Contexto institucional: Proyecto NATURCOR-IA	2
1.3. Objetivos del trabajo	3
2. Estado del arte	4
2.1. Modelos de lenguaje grandes (LLMs) y modelos de lenguaje pequeños (SLMs)	4
2.2. Alucinaciones en modelos generativos	6
2.3. Arquitectura RAG	7
2.3.1. RAG Naive	8
2.3.2. RAG Avanzado	8
2.3.3. RAG Modular	9
2.4. Bases de datos vectoriales y búsqueda semántica	10
2.5. Técnicas de segmentación documental (<i>chunking</i>)	12
2.6. Modelos de <i>embeddings</i>	13
2.7. Evaluación automática de sistemas RAG	15
3. Diseño e implementación	17
3.1. Arquitectura	17
3.2. Corpus documental	20
3.3. <i>Pipeline</i> de ingesta	21
3.3.1. Extracción de contenido	21
3.3.2. Segmentación	22
3.3.3. Generación de <i>embeddings</i>	24
3.3.4. Indexación	25
3.3.5. Gestión incremental	26

3.4. Componente conversacional	27
3.4.1. Memoria conversacional	27
3.4.2. Cadena RAG	31
3.5. Entorno de ejecución	33
4. Marco de evaluación	35
4.1. <i>Framework</i> de evaluación	35
4.2. Métricas	36
4.3. Protocolo experimental	38
4.4. Implementación	39
4.5. Análisis estadístico	40
5. Experimentación y resultados	43
5.1. Sistema de referencia	45
5.2. Tamaño de <i>chunk</i>	46
5.3. <i>Retriever</i> híbrido	48
5.4. <i>Multi-Query</i>	51
5.5. <i>Rewrite-Retrieve-Read</i>	54
5.6. Metadatos	58
5.7. Discusión global	60
6. Conclusiones y trabajo futuro	64
6.1. Conclusiones	64
6.2. Limitaciones	66
6.3. Líneas de trabajo futuro	67
Referencias bibliográficas	69
Anexo A. Repositorio de código y datos	74
Anexo B. <i>Prompts</i> del sistema	75
B.1. <i>Prompt</i> del sistema, versión inicial	75
B.2. <i>Prompt</i> del sistema, versión v2.2	75
B.3. <i>Prompt</i> de contextualización del <i>retriever</i>	77
B.4. <i>Prompt</i> de <i>Multi-Query</i>	77
B.5. <i>Rewrite-Retrieve-Read</i> , <i>AD</i>	78
B.6. <i>Rewrite-Retrieve-Read</i> , <i>GE</i>	78

Índice de tablas

3.1. Evolución del <i>prompt</i> del sistema entre las versiones v2.1 y v2.2. . . .	30
4.1. Resultados del test de normalidad de Shapiro-Wilk para las métricas evaluadas.	40
5.1. Configuraciones evaluadas y parámetros fijos del experimento.	44
5.2. Estadísticas descriptivas del sistema de referencia CS-1000.	46
5.3. Estadísticas descriptivas del experimento de tamaño de <i>chunk</i>	47
5.4. Estadísticas descriptivas del experimento de <i>retriever</i> híbrido.	50
5.5. Estadísticas descriptivas del experimento <i>Multi-Query</i>	53
5.6. Estadísticas descriptivas del experimento <i>Rewrite-Retrieve-Read</i>	56
5.7. Estadísticas descriptivas del experimento de inyección de metadatos. .	59
5.8. Resumen de las configuraciones evaluadas.	61

Índice de figuras

3.1. Arquitectura del sistema por componentes.	18
3.2. Diagrama de flujo del <i>pipeline</i>	19
3.3. Control del historial con ventana deslizante.	29
3.4. Flujo de la cadena RAG en la fase de generación.	32
5.1. Mecanismo del <i>retriever</i> híbrido.	49
5.2. Mecanismo de <i>Multi-Query retrieval</i>	52
5.3. Mecanismo de <i>Rewrite-Retrieve-Read</i>	55

Introducción

1.1. Contexto y motivación

En los últimos años, los asistentes conversacionales basados en inteligencia artificial han ganado presencia en numerosos sectores, cambiando la forma en que usuarios e instituciones acceden a la información (Gao et al., 2024). Un asistente conversacional es un sistema que interactúa en lenguaje natural con el usuario, interpreta sus consultas y devuelve respuestas también en lenguaje natural. Estos sistemas se basan en los modelos de lenguaje (*Language Models*, LM), redes neuronales entrenadas sobre grandes corpus textuales para predecir secuencias de palabras y generar texto coherente. La administración pública ofrece portales con información fiable sobre el turismo natural, como la Ventana del Visitante de los Espacios Naturales de Andalucía (Junta de Andalucía, 2026), y los asistentes conversacionales facilitan el acceso a este tipo de información, ya que el usuario puede formular consultas en lenguaje natural y recibir respuestas en formato conversacional, sin necesidad de localizar e interpretar los documentos originales. El sistema recupera la información y la reformula adaptándola a la consulta, reduciendo así la distancia conceptual entre la pregunta y el contenido relevante.

Sin embargo, cuando estos sistemas se aplican en un sector específico, surgen dos limitaciones principales. La primera son las «alucinaciones», es decir, la generación de respuestas creíbles pero falsas. La segunda es la ausencia de información: el modelo

solo dispone del conocimiento incorporado durante el entrenamiento y puede no tener acceso a información relevante para la consulta, aunque esta exista en otras fuentes. La arquitectura RAG (*Retrieval-Augmented Generation*) surge como una solución a estos problemas, combinando la recuperación semántica de información con la generación de lenguaje natural, permitiendo que las respuestas se apoyen en fuentes documentales concretas y reduciendo así la probabilidad de errores o invenciones (Lewis et al., 2020). En sectores públicos como el turismo natural, el problema se expande más allá del nivel técnico. Una respuesta incorrecta o un consejo falso sobre un itinerario o una normativa de acceso puede provocar un error del ciudadano con consecuencias críticas para su seguridad, el cumplimiento de la normativa y el prestigio de la institución que proporciona la información.

El uso de modelos de lenguaje pequeños (SLM, *Small Language Models*) de ejecución local (Touvron et al., 2023) evita el coste recurrente asociado al uso de servicios de inferencia externos e independiza el sistema de los cambios en las APIs y políticas de los proveedores comerciales. Además, estos modelos no requieren infraestructura de computación a gran escala, lo cual hace posible su despliegue en servidores convencionales.

1.2. Contexto institucional: Proyecto NATURCOR-IA

Este Trabajo Fin de Máster se encuadra en el proyecto NATURCOR-IA (referencia 2025/UCO008/032599), que forma parte del XI Plan Propio Galileo de la UCO (2025), en concreto de la Modalidad IA (Proyectos UCO-Transfiere). NATURCOR-IA es un proyecto de investigación y transferencia que busca mejorar la accesibilidad al patrimonio natural de la provincia de Córdoba aplicando tecnologías avanzadas de inteligencia artificial y procesamiento del lenguaje natural.

Como resultado principal del proyecto, se desarrollará una plataforma web de carácter interactivo con un asistente virtual multilingüe, basado en tecnologías de IA generativa, con el fin de facilitar el acceso a la información sobre itinerarios de senderismo y espacios naturales protegidos.

Para dar respuesta a esta necesidad, el presente Trabajo Fin de Máster propone el diseño, implementación y evaluación de un sistema conversacional basado en RAG con SLM de ejecución local, orientado a optimizar el acceso a la información de la plataforma de turismo natural de la Junta de Andalucía, analizando el impacto de los distintos parámetros de configuración sobre la calidad y fiabilidad de las respuestas generadas.

1.3. Objetivos del trabajo

El objetivo principal de este TFM es desarrollar y evaluar un sistema de asistencia conversacional basado en la arquitectura RAG y SLM de ejecución local, con el fin de optimizar el acceso a la información de la plataforma de turismo natural de la Junta de Andalucía.

Para alcanzarlo, se plantean los siguientes objetivos:

- Realizar un estudio previo sobre la aplicación de tecnologías de IA generativa y bases de datos vectoriales a la recuperación de información no estructurada.
- Analizar y comparar la idoneidad de las arquitecturas RAG para identificar la configuración más adecuada según los objetivos del sistema.
- Diseñar un *pipeline* de procesamiento documental que cubra las etapas de extracción de contenido (PDFs y web), segmentación (*chunking*), generación de *embeddings* e indexación vectorial.
- Desplegar un entorno de pruebas local basado en diferentes SLMs y arquitecturas RAG, configurando los distintos parámetros del sistema para realizar el análisis comparativo.
- Evaluar el rendimiento y la calidad de las respuestas generadas ante diferentes escenarios de consulta, analizando la reducción de alucinaciones y la precisión de la información.
- Extraer conclusiones sobre las configuraciones óptimas para este proyecto y proponer líneas de trabajo futuro.

Estado del arte

2.1. Modelos de lenguaje grandes (LLMs) y modelos de lenguaje pequeños (SLMs)

Los modelos de lenguaje son sistemas que procesan y generan texto en lenguaje natural. Su evolución no ha sido lineal: los primeros enfoques, basados en n-gramas, estimaban la probabilidad de cada palabra a partir de unas pocas palabras anteriores, lo que era suficiente para tareas simples pero insuficiente cuando la semántica requería un contexto más amplio. Las redes neuronales recurrentes (RNN) y sus variantes LSTM (*Long Short-Term Memory*) (Hochreiter y Schmidhuber, 1997) resolvieron en parte ese problema, pero introdujeron otro: el procesamiento era secuencial por naturaleza, lo que dificultaba el entrenamiento a escala y limitaba la generalización.

En 2017, Vaswani et al. publicaron la arquitectura Transformer con el mecanismo de atención (*self-attention*). A diferencia de las RNN, no procesa los *tokens* en secuencia: evalúa todos a la vez y decide cuánto peso dar a cada uno en relación con el resto. Eso resuelve el problema de las dependencias largas y, al mismo tiempo, permite paralelizar el entrenamiento. Desde entonces entrenar modelos sobre corpus masivos dejó de ser imposible. Todos los grandes modelos de lenguaje actuales se han construido sobre esta base arquitectónica.

Los *Large Language Models* (LLMs) son modelos basados en la arquitectura Transformer con decenas o centenas de miles de millones de parámetros, entrenados sobre corpus de texto de una escala difícil de dimensionar. Su desarrollo sigue tres fases: preentrenamiento sobre texto en bruto mediante predicción del siguiente *token*, ajuste fino supervisado sobre datos específicos de tarea (*fine-tuning*) y, a menudo, un paso de aprendizaje por refuerzo con retroalimentación humana (RLHF) para que las respuestas del modelo se ajusten a preferencias humanas (Ouyang et al., 2022). GPT-5, Claude y Gemini son los representantes más conocidos de esta familia y han mostrado un rendimiento sólido en tareas de comprensión y generación de texto en múltiples idiomas.

La interacción con un LLM se realiza a través del *prompt*, es decir, el texto de entrada que se le proporciona al modelo y a partir del cual genera su respuesta. El *prompt* suele estructurarse en dos partes: un *prompt* de sistema, que contiene las instrucciones que definen el comportamiento del modelo (rol, formato esperado, restricciones), y la entrada del usuario, que contiene la consulta concreta. La cantidad de texto que el modelo puede procesar en una sola interacción está limitada por su ventana de contexto (*context window*), medida en *tokens*, que abarca el *prompt* completo y la respuesta generada.

Además de la ventana de contexto, los LLMs tienen otras limitaciones significativas. Requieren *clusters* de GPUs de alto rendimiento, con los costes de adquisición y mantenimiento que eso conlleva. La mayoría se ofrecen como servicios en la nube, lo que implica que los datos salen de la infraestructura propia y pasan por terceros, generando también un gasto recurrente.

Como respuesta a estas restricciones, ha surgido el concepto SLM, que utilizan la misma arquitectura Transformer, pero con muchos menos parámetros, lo que permite ejecutarlos en servidores convencionales. Para conseguirlo sin perder demasiada calidad se emplean varias técnicas: la cuantización reduce la precisión numérica de los pesos del modelo, por ejemplo de 32 a 4 u 8 bits; el *pruning* elimina conexiones o neuronas con poco impacto en el resultado y la destilación del conocimiento entrena un modelo pequeño para que imite el comportamiento de uno mayor (Hinton et al., 2015). Llama 3.1 (Meta AI, 2024), Mistral 7B (Jiang et al., 2023) y Phi-3 (Abdin et al., 2024) son los representantes más conocidos de esta familia.

Irugalbandara et al. (2024) lo comprobaron en producción: evaluaron 29 diferentes SLMs de código abierto, incluyendo versiones cuantizadas a 2, 3 y 4 bits, como sustitución de GPT-4. Varios SLMs alcanzaron una calidad de respuesta comparable según evaluadores humanos. La latencia de la API de OpenAI varió entre 3,4 y 8,6 segundos a lo largo de 24 horas, mientras que los modelos ejecutados localmente se

mantuvieron estables. El coste bajó entre 5 y 29 veces. La brecha de calidad entre LLM y SLM existe, pero en dominios específicos se reduce hasta el punto de que las ventajas operativas de los modelos pequeños (coste, estabilidad y autonomía) acaban siendo definitivas.

2.2. Alucinaciones en modelos generativos

El término «alucinación» procede del campo de la visión por computadora, pero en el contexto de los modelos de generación de lenguaje natural (NLG) describe un problema concreto: el modelo produce texto fluido y gramaticalmente correcto, pero en realidad incorrecto o carente de validación objetiva (Ji et al., 2022). A diferencia de los errores sintácticos o de coherencia local, las alucinaciones son difíciles de identificar porque el texto generado suena convincente. El modelo formula afirmaciones con una seguridad que no se corresponde con su exactitud.

Con la expansión de los LLMs como sistemas de uso general, el problema ganó relevancia. En los modelos generativos basados en Transformer, todo el conocimiento está en los parámetros aprendidos durante el preentrenamiento. No hay ningún mecanismo interno que distinga entre una afirmación correcta y una inventada. El modelo genera la continuación más probable dada una entrada, sin acceso a fuentes externas ni capacidad de verificación (Huang et al., 2024). Las alucinaciones son, por tanto, una consecuencia directa de cómo funciona la generación, ya que el modelo genera la respuesta más probable, no la más verdadera.

Huang et al. (2024) identifican factores que contribuyen a las alucinaciones en tres etapas del ciclo de vida del modelo. En el preentrenamiento, los datos contienen información errónea, desactualizada o contradictoria que el modelo incorpora y memoriza sin filtro. En la fase de *fine-tuning*, si el modelo se entrena para responder siempre, sin posibilidad de expresar incertidumbre, acaba generando respuestas inventadas cuando la pregunta supera su conocimiento. En la fase de inferencia, el parámetro de temperatura controla el grado de aleatoriedad en la generación de los *tokens*, de manera que los valores altos aumentan la diversidad del texto a costa de la fidelidad.

Huang et al. (2024) proponen también una clasificación de dos dimensiones. La primera son alucinaciones de facticidad (*factuality hallucination*), es decir, el contenido contradice hechos verificables. La segunda trata de alucinaciones de fidelidad (*faithfulness hallucination*), lo que significa que la respuesta no sigue las instrucciones del usuario o se desvía del contexto dado.

Ji et al. (2022) establecen una clasificación diferente: alucinaciones intrínsecas y extrínsecas. Una alucinación es intrínseca cuando el contenido contradice una fuente de referencia disponible y extrínseca cuando se añade información que no se puede verificar con esa fuente.

El impacto de las alucinaciones depende del dominio. Por ejemplo, en consultas de carácter general, un dato incorrecto tiene pocas consecuencias. Sin embargo, en campos de alto riesgo como medicina o derecho, una alucinación puede llevar a decisiones erróneas con consecuencias reales (Ji et al., 2022; Huang et al., 2024). En el contexto de este trabajo, un asistente que proporcione información incorrecta sobre la longitud de un sendero, las condiciones de acceso o la normativa de un espacio protegido puede afectar directamente a la seguridad del usuario, al cumplimiento legal y al prestigio de la institución.

Los modelos generativos solo disponen de lo que aprendieron durante el preentrenamiento y no pueden verificar sus respuestas. Esta limitación es lo que promueve el uso de la arquitectura RAG, que se describe en la siguiente sección.

2.3. Arquitectura RAG

La arquitectura RAG responde a las dos limitaciones de los modelos generativos planteadas en las secciones anteriores: las alucinaciones, ya descritas en la sección 2.2, y la ausencia de información relevante, es decir, la incapacidad del modelo de responder a partir de datos que no formaban parte de su preentrenamiento. Los modelos generativos solo disponen de lo que aprendieron durante el preentrenamiento, sin acceso a fuentes externas ni capacidad de verificar sus propias respuestas. Lewis et al. (2020) propusieron juntar dos componentes que hasta entonces eran independientes: un sistema de recuperación de información (*retriever*) y un modelo generativo de lenguaje (*generator*). Antes de generar una respuesta, el *retriever* busca en una base de conocimiento externa los fragmentos más relevantes para la pregunta y los añade como contexto en el *prompt* del modelo. De esta manera, la respuesta se basa en información obtenida de documentos reales, en lugar de depender solamente de lo memorizado durante el entrenamiento, reduciendo así la probabilidad de alucinaciones.

Aun así, esta arquitectura sigue expuesta a los tipos de alucinaciones presentados en la sección 2.2. Por un lado, según la clasificación de Huang et al. (2024), ambos tipos siguen siendo posibles en un sistema RAG, ya que el modelo puede generar datos que no están en los documentos recuperados (facticidad) o ignorar las instrucciones del *prompt* (fidelidad). Por otro lado, según la clasificación de Ji et al. (2022), la distinción

tiene consecuencias prácticas: una contradicción con los fragmentos recuperados (intrínseca) se puede detectar, mientras que la introducción de información no presente en ellos (extrínseca) resulta más difícil de identificar.

Desde su publicación, la arquitectura RAG ha evolucionado notablemente. Gao et al. (2024) organizan esta evolución en tres niveles: RAG Naive (o básico), RAG Avanzado y RAG Modular. Los tres comparten la misma base, y cada nivel añade mecanismos para corregir las limitaciones del anterior.

2.3.1. RAG Naive

El RAG Naive parte de la propuesta original de Lewis et al. (2020): combinar un sistema de recuperación de información con un modelo generativo. Su *pipeline* se divide en tres fases (Gao et al., 2024).

La primera fase es la ingesta o indexación. Los documentos se segmentan en fragmentos de texto (*chunks*), cada uno se convierte en un vector numérico mediante un modelo de *embeddings* y se almacena en una base de datos vectorial junto con el texto original y, opcionalmente, metadatos como el nombre del documento o palabras clave. La segunda fase es la recuperación (*retrieval*). La pregunta del usuario se convierte también en un vector, y el sistema recupera de la base vectorial los k fragmentos más similares semánticamente (*top-k retrieval*). La tercera fase es la generación: los fragmentos recuperados se insertan en el *prompt* del modelo, generando la respuesta.

Esta arquitectura es fácil de implementar y suficiente para muchos escenarios, pero la calidad de los resultados depende por completo de la calidad de la recuperación: una consulta ambigua o formulada con terminología distinta a la del corpus puede recuperar fragmentos poco útiles o perder información relevante (Gao et al., 2024).

2.3.2. RAG Avanzado

El RAG Avanzado añade mejoras sobre la recuperación básica para superar las limitaciones del Naive. Gao et al. (2024) distinguen entre técnicas de *pre-retrieval*, que actúan antes de la búsqueda, y técnicas de *post-retrieval*, que filtran o reordenan los resultados.

Entre las técnicas de *pre-retrieval*, el RAG multi-query usa un modelo de lenguaje para reformular la consulta en varias versiones que enfatizan distintos aspectos (Gao et al., 2024). Cada versión lanza una búsqueda independiente, y los fragmentos obtenidos se combinan eliminando duplicados, lo que reduce la dependencia de la formulación exacta del usuario.

El enfoque *Rewrite-Retrieve-Read* (RRR), propuesto por Ma et al. (2023), funciona de otra manera: un modelo de lenguaje hace un paso previo a la búsqueda reformulando la consulta original para optimizarla (se entrena un modelo pequeño para esta tarea con aprendizaje por refuerzo). Esta idea puede combinarse con el enfoque iterativo descrito por Gao et al. (2024), en el que el sistema hace una recuperación inicial con la consulta original y después la reformula teniendo en cuenta los fragmentos ya recuperados, adaptándola al vocabulario del corpus. Por lo tanto, la segunda búsqueda produce resultados más precisos. Como evolución de este enfoque, Zhao et al. (2026) proponen unificar todo el proceso en una única arquitectura entrenada con aprendizaje por refuerzo, sin separación entre los pasos.

Entre las técnicas de *post-retrieval*, el *reranking* soluciona el problema de que no todos los fragmentos recuperados son igualmente relevantes. En lugar de usar los *top-k* fragmentos más cercanos semánticamente, se recupera un conjunto amplio y un modelo especializado evalúa la relevancia de cada uno respecto a la consulta (Nogueira y Cho, 2019). Solo los mejor puntuados se incluyen en el *prompt*, permitiendo al *retriever* inicial actuar con un criterio más amplio y al *reranker* seleccionar con mayor precisión.

El RAG híbrido combina búsqueda semántica por *embeddings* con búsqueda léxica mediante BM25 (Robertson y Zaragoza, 2009). Mientras que los *embeddings* capturan similitud conceptual, BM25 detecta coincidencias exactas de términos. La fusión de ambas señales, mediante técnicas como *Reciprocal Rank Fusion*, mejora los resultados cuando la consulta incluye términos técnicos o nombres propios que los *embeddings* no representan con suficiente precisión (Gao et al., 2024).

2.3.3. RAG Modular

El RAG Modular no es una arquitectura en concreto, más bien un paradigma de diseño: los componentes del *pipeline* (indexación, recuperación, generación, evaluación) son módulos intercambiables que pueden combinarse según los requisitos del sistema (Gao et al., 2024). Esto permite incorporar bucles de evaluación y retroalimentación en el proceso de generación.

El *Self-RAG*, propuesto por Asai et al. (2023), es el ejemplo más representativo. En lugar de ejecutar el *pipeline* de forma lineal, el modelo aprende a decidir cuándo buscar, evalúa la relevancia de los fragmentos recuperados y verifica si su propia respuesta se basa en el contexto. Para ello utiliza *tokens* especiales de reflexión (*reflection tokens*). Es un sistema más robusto frente a preguntas que no requieren recuperación externa o cuyos fragmentos recuperados son de baja calidad.

2.4. Bases de datos vectoriales y búsqueda semántica

Los sistemas de recuperación de información tradicionales (Elasticsearch, Apache Solr, búsqueda en bases de datos relacionales) utilizan búsqueda léxica: comparan términos exactos entre la consulta del usuario y los documentos indexados. El algoritmo BM25 es el representante más extendido de esta familia. Funciona bien cuando el usuario emplea el mismo vocabulario que aparece en los documentos, pero tiene una limitación ante diferencias de formulación: dos textos con el mismo significado pero distinto vocabulario producen puntuaciones bajas, mientras que dos textos que usan las mismas palabras clave pero tratan temas distintos pueden considerarse artificialmente «cercaños». Una consulta como «senderos para hacer con niños» no recuperaría documentos que hablen de «rutas de dificultad baja aptas para familias», aunque el contexto sea el mismo.

La búsqueda semántica se basa en una idea distinta. En lugar de simplemente comparar palabras, representa el significado. El texto, ya sea una consulta o un fragmento de documento, se convierte en un vector numérico de alta dimensión con un modelo de *embeddings*. Estos vectores están en un espacio matemático donde la distancia refleja similitud semántica, es decir, textos con significados parecidos se consideran cercaños entre sí incluso si no comparten ninguna palabra (Karpukhin et al., 2020). La similitud entre dos vectores se calcula mediante la similitud del coseno (Gao et al., 2024):

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} \quad (2.1)$$

El resultado varía entre -1 y 1, donde 1 es la máxima similitud. Esto permite recuperar documentos relevantes aunque la consulta del usuario esté formulada de forma completamente distinta a como está redactado el documento.

Las bases de datos vectoriales están diseñadas para almacenar, indexar y consultar vectores de alta dimensión de forma eficiente. Las bases de datos relacionales convencionales están optimizadas para búsquedas por clave o comparaciones escalares. Sin embargo, la búsqueda semántica requiere encontrar los k vectores más similares a una consulta (*k-nearest neighbor search*, kNN) entre millones de entradas. Una búsqueda exacta en un espacio de alta dimensión no es viable en tiempo real, ya que tiene coste $O(n \cdot d)$, donde n es el número de vectores y d su dimensión.

Las bases de datos vectoriales resuelven este problema usando los índices ANN (*Approximate Nearest Neighbor*). Estos índices aproximan el resultado sacrificando una parte de precisión, aumentando así la velocidad (Johnson et al., 2017).

Los dos algoritmos de indexación más conocidos son HNSW e IVF.

- HNSW (*Hierarchical Navigable Small World*), propuesto por Malkov y Yashunin (2020), construye un grafo jerárquico de varias capas. Las superiores tienen pocos nodos con conexiones de largo alcance para moverse rápido hacia la zona de interés y las inferiores son más densas y permiten afinar de manera local. La búsqueda empieza arriba, y desciende capa a capa y termina en los vecinos más cercanos sin explorar el índice completo. Es el algoritmo habitual si el índice cabe en memoria.
- IVF (*Inverted File Index*) divide el espacio vectorial en clústeres usando k-means y durante la búsqueda examina solo los más próximos a la consulta. A diferencia de HNSW, escala mejor a colecciones muy grandes y puede combinarse con cuantización de producto (*Product Quantization*, PQ) para comprimir los vectores y reducir el uso de memoria (Johnson et al., 2017).

Entre las de código abierto, FAISS (*Facebook AI Similarity Search*) es una opción de referencia, desarrollada por Meta AI, que implementa IVF, HNSW, PQ y combinaciones de estos algoritmos. FAISS está optimizada para GPU y CPU, y escala a miles de millones de vectores (Johnson et al., 2017). Funciona integrada en el código de la aplicación, sin servidor ni API REST propios. Chroma es una alternativa más reciente orientada a aplicaciones LLM, con una API sencilla y soporte nativo para filtrado por metadatos (Chroma, 2026). Qdrant ofrece un servidor vectorial completo con filtrado avanzado por atributos de los documentos (Qdrant, 2026).

Entre los servicios en la nube, Pinecone y Weaviate gestionan la infraestructura del índice de forma externa (Pinecone, 2026; Weaviate, 2026). Eliminan la necesidad de configurar y mantener el servidor de manera local, pero los datos salen de la infraestructura propia, lo que es una restricción relevante en entornos con información sensible.

En sistemas RAG, la elección de la base de datos vectorial afecta a la latencia de recuperación y a la escalabilidad (Gao et al., 2024). En la práctica, los algoritmos ANN alcanzan más de 95 % de precisión con tiempos de respuesta inferiores a 10 ms (Johnson et al., 2017; Malkov y Yashunin, 2020), lo que los hace una opción adecuada para aplicaciones en tiempo real.

2.5. Técnicas de segmentación documental (*chunking*)

Los modelos de *embeddings* tienen una ventana de contexto limitada, lo que hace imposible procesar un documento completo de una vez (Gao et al., 2024). Aun así, incluso cuando el documento cabe en ese límite, representar grandes fragmentos de texto en un único vector produce el resultado demasiado genérico. La información relevante para una consulta en concreto se pierde, y la recuperación empeora (Karpukhin et al., 2020).

La segmentación documental, o *chunking*, divide los documentos en fragmentos de un número determinado de *tokens* (p.j. 100-512 *tokens*) antes de generar los *embeddings*. Cada fragmento se indexa por separado, lo que permite al *retriever* localizar con más precisión qué parte del documento responde a la pregunta del usuario (Lewis et al., 2020).

Existen dos parámetros principales que controlan la segmentación: el `chunk_size`, que fija el tamaño máximo de cada fragmento (en caracteres o *tokens*), y el `chunk_overlap`, que define cuántas unidades del final de un fragmento se repiten al inicio del siguiente. El solapamiento evita que la segmentación corte una idea a la mitad y la pierda entre dos fragmentos.

La elección de estos valores influye en la calidad de la recuperación. Con fragmentos pequeños, las representaciones son más precisas, pero pueden perder el contexto necesario para responder preguntas cuyas respuestas están distribuidas por varios párrafos. Al contrario, con fragmentos grandes el contexto se conserva, pero el vector representa un contenido menos específico y la coincidencia con la consulta es menos probable. No existe una configuración óptima universal; todo depende del tipo de documentos, del tipo de consultas y del modelo de *embeddings* (Gao et al., 2024).

Se distinguen tres estrategias de segmentación:

- La segmentación de tamaño fijo (*fixed-size chunking*) divide el texto en fragmentos de longitud determinada, con solapamiento opcional. Es la más sencilla de implementar y la más extendida en producción. Herramientas como `LangChain` la implementan con `RecursiveCharacterTextSplitter`, que intenta dividir por separadores naturales (párrafos, puntos, espacios) antes de cortar por longitud bruta, lo que reduce la probabilidad de partir frases a mitad.
- La segmentación estructural (*structural chunking*) divide el texto según su estructura. Respeta las jerarquías del documento como secciones definidas por

encabezados, párrafos, listas o, en PDFs con estructura lógica marcada, elementos del árbol del documento (Gao et al., 2024). Cada fragmento corresponde a una unidad real del texto, por eso la coherencia semántica está garantizada por el propio documento. El problema es que esto solo funciona si el documento está bien estructurado, y muchos PDFs obtenidos por escáner o con formatos no estándar no lo están.

- La segmentación semántica (*semantic chunking*) divide el texto según cambios de contenido. Se agrupan oraciones con representaciones vectoriales similares y se cortan cuando la similitud cae por debajo de un umbral. Kamradt (2023) formalizó este enfoque, que más tarde fue adoptado en *LangChain*. Es la estrategia más precisa en teoría, pero también la más costosa, ya que hay que calcular *embeddings* para cada oración antes de saber dónde cortar.

El *chunking* es uno de los factores que más afecta a la calidad de la recuperación. Un sistema puede tener el documento correcto en su índice y aun así devolver fragmentos irrelevantes si la segmentación fue inadecuada, o perder información clave distribuida por varios fragmentos que el *retriever* no recupera juntos. Barnett et al. (2024) y Gao et al. (2024) lo sitúan entre los factores de mayor impacto en el rendimiento RAG, junto con el modelo de *embeddings* y la configuración del *retriever*.

2.6. Modelos de *embeddings*

Un modelo de *embeddings* transforma texto en un vector numérico de alta dimensión, donde la distancia entre vectores refleja la similitud semántica entre los textos originales, tal como está descrito en la sección anterior.

Los primeros modelos de *embeddings*, como Word2Vec (Mikolov et al., 2013) y GloVe (Pennington et al., 2014), asignaban un vector fijo a cada palabra. La limitación principal consistía en que una misma palabra, por ejemplo, «banco» tenía la misma representación sin importar el contexto. Esto limitaba su utilidad en la búsqueda semántica, donde el contexto determina el significado.

La introducción de BERT (Devlin et al., 2018) superó esta limitación con atención bidireccional, es decir, cada *token* recibe una representación que depende del contexto. Sin embargo, BERT procesa pares de textos conjuntamente (*cross-encoder*), lo que complica la búsqueda a gran escala. Además, sus representaciones no están optimizadas para comparar fragmentos de texto por similitud (Reimers y Gurevych, 2019).

En 2019, Reimers y Gurevych propusieron Sentence-BERT (SBERT), una modificación basada en redes siamesas, que entrena el modelo con pares de textos similares y disimilares (*contrastive learning*), así los textos con el mismo significado quedan cerca en el espacio vectorial. Este enfoque permite precalcular los *embeddings* de los documentos e indexarlos en una base vectorial, de modo que cada consulta solo requiere codificar la pregunta y buscar los vectores más cercanos. Actualmente, este tipo de entrenamiento es el que se aplica en la mayoría de los modelos de recuperación semántica utilizados en RAG.

Los *embeddings* producidos por los modelos descritos en esta sección son densos. Se trata de vectores compactos donde todas las dimensiones tienen valores distintos de cero, optimizados para capturar similitud semántica incluso cuando el vocabulario de la consulta difiere del de los documentos.

Asimismo, los modelos de *embeddings* se diferencian por arquitectura, cobertura lingüística y dominio de uso. Por arquitectura, los modelos *encoder-only* como BERT y sus derivados son los más habituales en tareas de recuperación: procesan el texto de forma bidireccional y están diseñados para producir representaciones. Los modelos *decoder-only* (GPT, Llama) también pueden usarse para generar *embeddings* extrayendo activaciones de capas intermedias, aunque no están optimizados para ello en su configuración base y el rendimiento suele ser peor. Sin embargo, con ajuste fino específico (*contrastive fine-tuning*) alcanzan resultados comparables.

Por cobertura lingüística, los modelos monolingües son más precisos dentro de su idioma. Los multilingües, como LaBSE (Feng et al., 2020) o multilingual-E5 (Wang et al., 2024), representan textos en distintos idiomas en un espacio vectorial común. Aparte de modelos de propósito general, también existen modelos ajustados a dominios específicos como medicina, derecho o código, entre otros.

Entre las familias más usadas en sistemas RAG están E5 (Wang et al., 2022), BGE (BAAI, 2023) y **sentence-transformers**, que reúne múltiples variantes con distintos compromisos entre rendimiento y tamaño.

Para comparar modelos, se usa el *benchmark* de referencia - MTEB (*Massive Text Embedding Benchmark*, Muennighoff et al., 2022). Este modelo evalúa los modelos en tareas de recuperación, clasificación semántica y búsqueda de similitud sobre datos en varios idiomas. Sus tablas de clasificación suelen ser el punto de partida para elegir modelo según el idioma y el tipo de corpus.

2.7. Evaluación automática de sistemas RAG

La evaluación de sistemas RAG se enfrenta a un obstáculo que no existe en los sistemas de recuperación clásica. La respuesta generada no necesita coincidir textualmente con ninguna respuesta de referencia para ser correcta. Las métricas tradicionales como BLEU (Papineni et al., 2002) o ROUGE (Lin, 2004) comparan salida y referencia por coincidencia léxica. Este enfoque no funciona en sistemas generativos, donde la misma pregunta puede tener respuestas equivalentes con vocabulario totalmente distinto. Además, en un sistema RAG la calidad depende también de si el contexto recuperado es relevante y si la respuesta se basa en él, aspectos que las métricas léxicas no miden.

Como alternativa a los métodos tradicionales, ha surgido el enfoque *LLM-as-judge* (Zheng et al., 2023). Un modelo de lenguaje evalúa la respuesta de forma semántica en lugar de comparar cadenas de texto. Esto permite evaluar sistemas generativos sin *ground truth*, lo que elimina la necesidad de crear un dataset de referencia.

Sobre este paradigma se han construido varios *frameworks* de evaluación que automatizan el cálculo de métricas para sistemas RAG. TruLens (TruEra, 2026) es un *framework* que aplica el paradigma LLM-as-judge a la evaluación de sistemas RAG, agrupando sus métricas en la RAG Triad. La primera, *context relevance*, evalúa cada fragmento recuperado por separado según su utilidad para responder a la pregunta; el resultado final es el promedio. La segunda, *groundedness*, descompone la respuesta en afirmaciones atómicas y verifica cuántas se basan en el contexto, usando un modelo NLI o un LLM. La tercera, *answer relevance*, comprueba si la respuesta aborda la pregunta formulada sin depender del contexto. Principalmente, TruLens está pensado para monitorización en producción y funciona más con LLMs externos.

DeepEval (Confident AI, 2026) aplica un enfoque similar al de *TruLens*, usando también un LLM como juez, pero amplía el conjunto de métricas y añade un campo *reason* en cada puntuación, donde el modelo justifica en lenguaje natural su decisión. Una de las métricas es *faithfulness*, que descompone la respuesta en afirmaciones y verifica si se basa en el contexto. La siguiente, *contextual relevancy*, mide qué porcentaje de afirmaciones recuperadas contiene información útil. La última métrica de este enfoque es *contextual recall*, que evalúa si el contexto cubre los aspectos necesarios para responder. *DeepEval* puede integrarse en *pipelines* de integración continua y usa habitualmente APIs externas para el modelo evaluador.

ARES (*Automated RAG Evaluation System*, Saad-Falcon et al., 2023) no usa un LLM grande como juez durante la evaluación. En su lugar, entrena clasificadores ligeros basados en DeBERTa sobre datos sintéticos generados previamente por un

modelo grande, con tres clasificadores independientes para relevancia del contexto, fidelidad de la respuesta y relevancia de la respuesta. Para controlar el sesgo de los datos sintéticos, aplica *Prediction-Powered Inference* (PPI), que requiere alrededor de 150 muestras anotadas manualmente para la calibración. Una vez entrenados, los clasificadores se ejecutan de manera local con menor coste computacional, pero el proceso de preparación requiere *ground truth* y una fase de entrenamiento que los otros *frameworks* no necesitan.

A diferencia de TruLens y *DeepEval*, RAGAS (*Retrieval-Augmented Generation Assessment*, Es et al., 2023) fue diseñado originalmente para evaluar *pipelines* RAG sin depender de respuestas de referencia (*reference-free*), es decir sin *ground truth*. Aunque sus versiones actuales han ampliado el conjunto de métricas e incluyen también algunas que requieren *ground truth*, el *framework* mantiene un enfoque *reference-free*, lo que lo hace especialmente adecuado para escenarios en los que no se dispone de respuestas anotadas. Sus métricas se aplican sobre muestras individuales de pregunta-contexto-respuesta. *Faithfulness* mide qué porcentaje de las afirmaciones extraídas de la respuesta se basan en el contexto recuperado. Otra métrica es *Answer Relevance*, que calcula la similitud semántica entre la pregunta original y preguntas hipotéticas generadas a partir de la respuesta. Además, en sus versiones actuales, el *framework* añade métricas de recuperación como *Context Precision* (Ragas, 2026), que evalúa la calidad del ranking verificando si los fragmentos relevantes aparecen en las primeras posiciones. En su variante sin referencia (también denominada *Context Utilization*), la relevancia de cada fragmento se determina comparándolo con la respuesta generada en lugar de con una respuesta de referencia.

Diseño e implementación

El capítulo anterior ha presentado los componentes principales para construir un sistema RAG: la arquitectura (sección 2.3), la base de datos vectorial (sección 2.4), la estrategia de segmentación (sección 2.5), el modelo de *embeddings* (sección 2.6) y el marco de evaluación (sección 2.7). Cada uno admite varias alternativas, y su elección determina el comportamiento del sistema y sus posibles limitaciones. Este capítulo presenta las decisiones tomadas en este trabajo para cada uno de estos elementos.

3.1. Arquitectura

El sistema se basa en la arquitectura RAG Naive descrita en la sección 2.3, a la que se añade memoria conversacional. La elección de esta configuración se justifica por dos motivos principales. En primer lugar, al ser la arquitectura inicial propuesta por Lewis et al. (2020), RAG Naive actúa como el modelo base con el que comparar técnicas más avanzadas. En segundo lugar, su simplicidad arquitectónica permite aislar el efecto de cada componente del *pipeline* sin que las interacciones con técnicas adicionales distorsionen los resultados. Las técnicas de RAG Avanzado descritas en la sección 2.3 (*retriever* híbrido, *multi-query* y *rewrite-retrieve-read*) se incorporan posteriormente en el capítulo 5 como variantes experimentales sobre esta configuración de referencia.

La figura 3.1 muestra los componentes principales del sistema (interfaz de chat, orquestador `rag_chain`, memoria conversacional, `history-aware` retriever, módulo de recuperación, módulo de generación) y sus conexiones con el servicio de modelos Ollama y el índice vectorial FAISS.

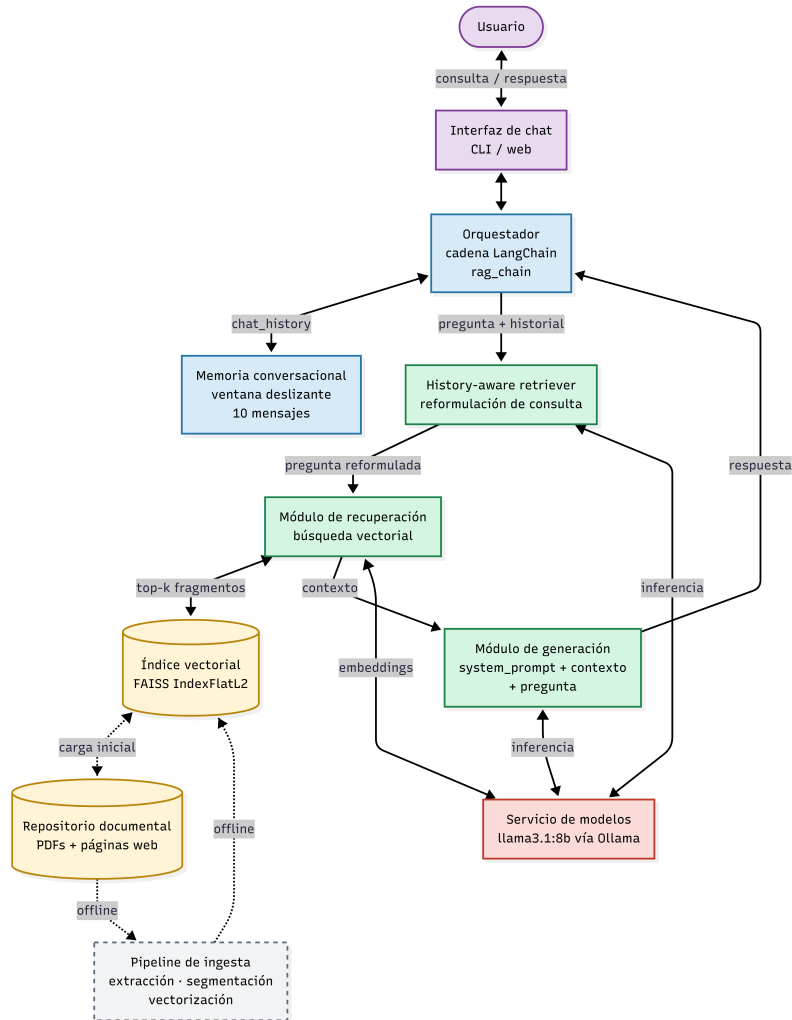


Figura 3.1. Arquitectura del sistema por componentes.

El sistema se organiza en la fase de indexación, que procesa los documentos antes de que el sistema reciba cualquier consulta, y la fase de generación, que responde a las preguntas del usuario en tiempo real. La figura 3.2 muestra el flujo de ambas fases.

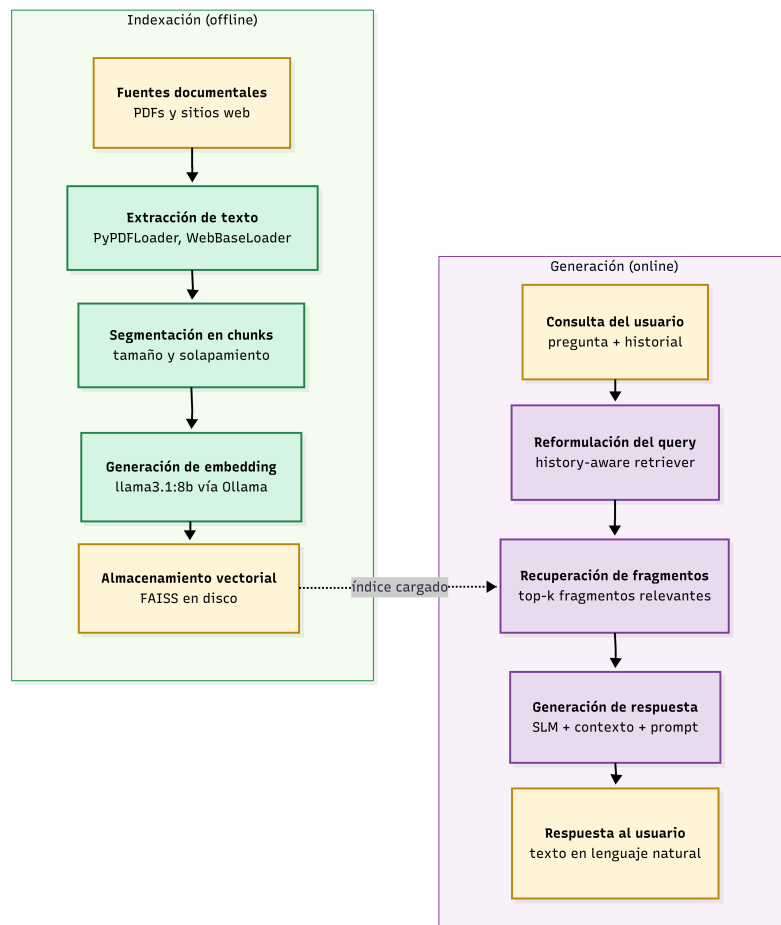


Figura 3.2. Diagrama de flujo del *pipeline*.

En el diseño de sistemas de aprendizaje automático, una operación se considera *offline* cuando puede ejecutarse de forma anticipada, sin depender de las entradas del usuario, y su resultado puede almacenarse para reutilizarlo después. Al contrario, una operación es *online* cuando ocurre directamente como respuesta a una solicitud en tiempo real. Esta distinción aplica, aunque el sistema sea completamente local, ya que la vectorización de los documentos puede realizarse en cualquier momento antes de la primera consulta y el índice resultante queda guardado en disco. La generación de la respuesta, en cambio, no puede anticiparse porque depende de la pregunta concreta del usuario.

La fase de indexación convierte los documentos del corpus en un índice de búsqueda. El corpus se basa en documentos PDF descargados del portal oficial y páginas web del mismo. El texto extraído de cada fuente se divide en fragmentos de tamaño y solapamiento configurables. Cada fragmento se vectoriza con el modelo de *embeddings* llama3.1:8b ejecutado localmente a través de Ollama, la decisión se justifica en la sección 3.3.3. Los vectores se almacenan en un índice FAISS en disco junto con el texto original de cada fragmento y su información básica. Para no

procesar los mismos archivos una y otra vez, el sistema mantiene un registro de los documentos ya indexados, lo que permite añadir nuevas fuentes de forma incremental sin reconstruir el índice desde cero.

La fase de generación responde a las consultas en el momento en que se producen. Cuando el usuario envía una pregunta, el sistema sigue tres pasos. Primero, un componente de recuperación consciente del historial reformula la pregunta para obtener una consulta autónoma que no requiera conocer el contexto de intercambios anteriores, como se describe en detalle en la sección 3.4.1. Segundo, la consulta reformulada se vectoriza y se compara con el índice FAISS para recuperar los k fragmentos más relevantes. Tercero, el modelo llama3.1:8b recibe como entrada el *prompt* del sistema, la pregunta del usuario y los fragmentos recuperados, y genera la respuesta. El historial de la conversación está limitado a los diez mensajes más recientes para controlar el tamaño total del contexto y mantener una latencia de respuesta aceptable en el entorno de ejecución local.

El sistema admite dos modos de ejecución. El modo chat permite la interacción directa con el usuario y es el que se describe en esta sección. El modo test, utilizado en la fase experimental, ejecuta las preguntas de un fichero externo de forma independiente y se describe en la sección 4.4.

El *pipeline* se desarrolla en Python con **LangChain**, una biblioteca que integra los cargadores de documentos, el divisor de texto, la conexión con Ollama y FAISS, y las cadenas que unen la recuperación con la generación de respuestas.

3.2. Corpus documental

El corpus está formado principalmente por las fichas oficiales de senderos de la Red de Senderos de Andalucía, publicadas por la Junta de Andalucía en el portal Ventana del Visitante (juntadeandalucia.es/medioambiente). Es la fuente oficial y pública de información sobre rutas en los espacios naturales protegidos de Andalucía. Su estructura uniforme entre documentos simplifica el procesamiento. Los enlaces se recopilaron directamente del portal y los archivos se descargaron de forma automatizada, obteniendo un total de 671 documentos PDF.

Cada documento corresponde a una ruta y la mayoría de ellos tiene una estructura fija de tres páginas. La primera recoge los datos técnicos del recorrido como provincia, municipios, longitud, tiempo estimado, dificultad, tipo de trayecto, desnivel y coordenadas. La segunda contiene una descripción de flora, fauna y puntos de interés. La tercera presenta el perfil altimétrico y el mapa. La extracción del texto se realiza con **PyPDFLoader**, que procesa el documento página por página. La tercera

página es un elemento gráfico sin texto extraíble, así que a continuación se usa solo con el contenido de las dos primeras. Por la misma razón, de los documentos completamente gráficos del corpus, como escaneos o mapas sin capa de texto, no se extrae el contenido.

El corpus abarca las ocho provincias de Andalucía. Junto a las fichas estándar hay también guías de rutas de largo recorrido divididas por etapas, con formato y extensión distintos. Una parte de los documentos está disponible también en inglés. Las versiones en inglés que se descargaron se indexan como documentos independientes, sin vinculación directa con sus equivalentes en español. Este trabajo no aborda el soporte multilingüe de los *embeddings*, y, por tanto la búsqueda puede estar afectada cuando la consulta y los fragmentos no están en el mismo idioma. La sección 6.3 retoma este punto como línea de trabajo futuro. Además, el corpus es estático, es decir, no se actualiza cuando el portal añade nuevos documentos. Dado que la información sobre senderos puede cambiar (condiciones de acceso, restricciones), esta es una limitación del sistema. El mecanismo de gestión incremental descrito en la sección 3.3.5 permite añadir documentos nuevos al índice sin reconstruirlo entero, pero la descarga de los PDFs desde el portal y la detección de cambios en documentos ya indexados quedan fuera del alcance de este TFM y se recogen en la sección 6.3.

Como fuente complementaria se cargaron dos páginas web del portal con **WebBaseLoader**. El análisis del contenido extraído mostró que ambas son páginas de navegación con poco texto útil para consultas sobre rutas. El portal sí tiene el contenido descriptivo útil, por ejemplo, el buscador de rutas y las páginas individuales de cada itinerario incluyen información detallada sobre cada recorrido. Sin embargo, esas páginas HTML no pueden recorrerse de forma automatizada, ya que cada sendero tiene su propia URL generada dinámicamente, y no hay un índice estático que permita recopilarlas fácilmente. Las fichas PDF, en cambio, están disponibles en una página del portal que contiene los enlaces directos a cada archivo, lo que permite obtener la lista completa de URLs y descargarlos automáticamente. Como cada PDF contiene toda la información técnica y descriptiva de una ruta, estos documentos se eligen como la fuente principal del corpus.

3.3. *Pipeline* de ingesta

3.3.1. Extracción de contenido

La primera etapa del *pipeline* es la extracción del contenido. Antes de que el sistema pueda responder consultas, los documentos deben estar cargados, procesados

y convertidos en fragmentos de texto listos para la vectorización. El sistema tiene dos tipos de fuentes que se procesan de forma distinta: páginas web y documentos PDF.

Las páginas web se cargan con `WebBaseLoader` de `LangChain`, que hace una petición HTTP, descarga el HTML y extrae el texto visible descartando etiquetas y elementos de navegación. Para las peticiones, se utiliza un User-Agent de navegador estándar para evitar bloqueos del servidor. Como se menciona en la sección 3.2, el contenido extraído de las dos páginas del portal era principalmente navegacional, con poco texto útil para consultas sobre rutas. Esta fuente aporta poco al corpus; su inclusión responde al diseño inicial del sistema, antes de analizar en detalle la arquitectura del portal.

Los PDFs son la fuente principal. Los enlaces a los archivos se recopilaban del portal Ventana del Visitante y se almacenaron en un fichero Excel que actúa como índice del corpus. A partir de esta lista, el sistema descarga los archivos con la librería `requests` de Python y los guarda en una carpeta local. Antes de descargar cada archivo, el sistema comprueba si ya existe en disco, es lo que permite relanzar la extracción sin volver a descargar archivos ya presentes.

La extracción del texto se hace con `PyPDFLoader` de `LangChain`, que recorre cada PDF página a página y devuelve un objeto independiente por página. Esto permite controlar con precisión qué contenido se indexa. La tercera página de cada ficha es el perfil de elevación y el mapa. Al ser un elemento gráfico sin capa de texto, no produce contenido al procesarse y el sistema trabaja en la práctica solo con las dos primeras. Lo mismo ocurre con los documentos completamente gráficos del corpus, como escaneos o mapas sin texto.

Sin embargo, `PyPDFLoader` permite extraer contenido de imágenes mediante OCR (*Optical Character Recognition*, reconocimiento óptico de caracteres) con el parámetro `extract_images`, que por defecto está desactivado. En este proyecto se mantiene desactivado, ya que pasar un mapa de ruta o un perfil de elevación por OCR produce texto fragmentado y no estructurado (coordenadas sueltas, fragmentos de curvas de nivel, leyendas a medias) que entraría en el índice sin aportar nada útil al modelo. Este tipo de ruido contextual se considera uno de los factores que degradan la recuperación (Gao et al., 2024).

Al completar la extracción, el texto obtenido pasa a la segmentación en *chunks*.

3.3.2. Segmentación

Una vez extraído el texto, el *pipeline* lo divide en fragmentos más pequeños antes de pasar a la vectorización. Como se describe en la sección 2.5, hay tres estrategias principales para hacerlo: segmentación de tamaño fijo, estructural y semántica. En

este sistema se utiliza la segmentación de tamaño fijo. La segmentación estructural se descartó por las características del corpus. La mayoría de las fichas de la Junta de Andalucía son trípticos con el texto dividido en varias columnas, sin etiquetas estructurales que marquen una jerarquía lógica de secciones. Además, tienen solo dos páginas de texto, por lo que un corte estructural produciría fragmentos demasiado pequeños para conservar el contexto necesario para la recuperación. La segmentación semántica se descartó porque calcula *embeddings* por oración antes de cortar, lo que multiplica considerablemente el número de llamadas al modelo de *embeddings* en la fase de indexación. La segmentación de tamaño fijo no tiene ninguna de estas restricciones y es la más documentada en la literatura sobre sistemas RAG (Gao et al., 2024; Barnett et al., 2024).

Dentro de **LangChain**, la clase utilizada es **RecursiveCharacterTextSplitter**. A diferencia de **CharacterTextSplitter**, que corta directamente por longitud, esta clase primero divide por separadores jerárquicos (párrafos, saltos de línea, espacios), lo que reduce los cortes a mitad de oración (LangChain, 2024d). Las alternativas basadas en tokenización añaden una dependencia de un tokenizador concreto, lo que haría que los límites de los fragmentos variaran según el modelo utilizado.

La unidad de medida elegida son los caracteres y no los *tokens*. A diferencia de los recuentos por *tokens*, que dependen del tokenizador de cada modelo, los caracteres producen el mismo resultado independientemente del SLM utilizado, lo que facilita la reproducción de los experimentos sin modificar los parámetros de segmentación.

Para la configuración base se fijaron `chunk_size=1.000` y `chunk_overlap=200`. El intervalo de 100-512 *tokens* aparece en Gao et al. (2024) y Barnett et al. (2024) como punto de partida habitual en sistemas RAG, y la documentación de **LangChain** usa 1.000 caracteres (aproximadamente 250 *tokens*) como valor de ejemplo en su guía de text splitting (LangChain, 2024b). Atendiendo a la estructura del corpus, cada ficha de sendero contiene entre 3.000 y 4.000 caracteres de texto útil por página, por lo que con `chunk_size=1.000` cada página se divide en tres o cuatro fragmentos que conservan la coherencia temática interna. El solapamiento de 200 caracteres (el 20 % del tamaño del fragmento) evita que un corte elimine el contexto necesario para interpretar el inicio del siguiente fragmento sin duplicar en exceso el contenido indexado.

Esta configuración también depende del parámetro k del *retriever*, fijado en 10 en el sistema de referencia. El producto `chunk_size * k = 1.000 * 10` determina el volumen de contexto total que se inyecta en cada *prompt* (aproximadamente 10.000 caracteres), lo que a su vez condiciona la latencia y el rendimiento del modelo. Para

comparar configuraciones de forma justa esta relación se tuvo en cuenta en el diseño de los experimentos de la sección 5.2

Cada fragmento resultante es un objeto `Document` de `LangChain` con dos campos: `page_content`, que contiene el texto, y `metadata`, que hereda de `PyPDFLoader` el nombre del archivo de origen (*source*) y el número de página (*page*). Estos metadatos se conservan en el índice FAISS y permiten identificar, para cada respuesta generada, qué documentos concretos contribuyeron al contexto. Esta información se registra en los resultados de los experimentos y también se muestra en el modo chat. Tras procesar el corpus completo, el índice FAISS contiene 11.927 fragmentos.

3.3.3. Generación de *embeddings*

En esta etapa, los fragmentos de texto producidos en la segmentación se transforman en vectores numéricos. Cada *chunk* queda representado por un vector de alta dimensión que refleja su contenido semántico, es decir, fragmentos con significados parecidos ocupan posiciones próximas en el espacio vectorial. Esta representación es lo que permite comparar matemáticamente la consulta del usuario con los documentos indexados mediante similitud del coseno, como se describe en la sección 2.4.

Para la vectorización se usa llama3.1:8b, ejecutado de manera local a través del servidor Ollama. La elección del modelo se justifica en la sección 3.5. La idea es mantener la consistencia experimental: con el mismo modelo para vectorizar el corpus y para generar respuestas, la variabilidad entre componentes se reduce y el efecto de las demás variables queda más aislado en los experimentos. Como se indica en la sección 2.6, los modelos *decoder-only* sin ajuste fino específico no están optimizados para la tarea de *embeddings*. Las alternativas *encoder* como BGE (BAAI, 2023) o E5 (Wang et al., 2022) dan en general mejores resultados en recuperación semántica. Esta es una limitación conocida del sistema base. Reemplazar el modelo de *embeddings* por un *encoder* especializado es la primera mejora directa por explorar, y se recoge como tal en la sección 6.3.

La vectorización opera sobre el texto de cada *chunk*, concretamente sobre el campo `page_content` que produce el divisor de texto. La clase `OllamaEmbeddings` de `LangChain` gestiona la comunicación con el servidor Ollama mediante peticiones HTTP autenticadas. Cada *chunk* se procesa de manera individual y su vector resultante, de dimensión 4.096 se incorpora al índice FAISS. Como la vectorización fue la fase más lenta del *pipeline* en el entorno local se implementó el procesamiento por lotes y el guardado periódico del índice, que se describen en la sección 3.3.5.

La misma operación se aplica en el momento de la consulta, es decir, cuando el usuario envía una pregunta, esta se vectoriza con el mismo modelo antes de

lanzar la búsqueda en el índice. El modelo de *embeddings* tiene que ser el mismo en la fase de indexación y en la de inferencia. En el caso contrario, los vectores de documentos y consultas corresponden a espacios distintos y la búsqueda por similitud no produce resultados válidos. El uso de llama3.1:8b en ambas etapas garantiza que la comparación sea coherente.

3.3.4. Indexación

Con los vectores generados en la etapa anterior, el *pipeline* los organiza en una estructura que permite buscar, ante cualquier consulta, los fragmentos más próximos sin recorrer el corpus completo. La sección 2.4 describe los distintos tipos y sus propiedades desde un punto de vista teórico. En esta sección se concreta qué índice construye el sistema y qué implica eso en la práctica.

Como base de datos vectorial se eligió FAISS, desarrollada por Meta AI (Johnson et al., 2017). Como se analiza en la sección 2.4, las alternativas se dividen entre servicios gestionados en la nube como Pinecone y Weaviate, y soluciones de despliegue local como FAISS, Chroma y Qdrant. Los servicios en la nube se descartaron por el carácter experimental del sistema, ya que la dependencia de un servicio externo añade costes recurrentes y condiciona la reproducibilidad. Entre las opciones locales, Chroma y Qdrant tienen que ejecutarse como un servicio externo al que el sistema se conecta. FAISS, en cambio, se integra directamente como librería en el código de la aplicación, sin servidor ni API REST propios, lo que simplifica el despliegue.

El tipo de índice que **LangChain** construye al llamar a `FAISS.from_documents()` es `IndexFlatL2`. A diferencia de los índices aproximados como HNSW e IVF descritos en la sección 2.4, este índice realiza búsqueda exacta. Para cada consulta calcula la distancia euclídea entre el vector de la pregunta y todos los vectores del índice, con un coste lineal en el número de fragmentos. Al no aplicar ninguna aproximación, los k fragmentos devueltos son siempre los k más cercanos al vector de consulta en términos de distancia L2. Los índices aproximados como HNSW e IVF están diseñados para colecciones de decenas de millones de vectores (Johnson et al., 2017), y con un corpus de 11.927 fragmentos la búsqueda exacta resulta suficiente.

El índice en disco se compone de dos ficheros. El fichero `index.faiss` almacena los vectores en formato binario, con un tamaño total de 186 MB en la configuración baseline. El fichero `index.pkl` almacena, para cada fragmento, el texto original junto con sus metadatos (nombre del archivo fuente y número de página, descritos en la sección 3.3.2). **LangChain** mantiene la correspondencia entre ambos ficheros, de modo que una búsqueda devuelve directamente el objeto `Document` con su contenido y sus metadatos.

El índice se persiste en disco mediante `save_local()` y se recupera en sesiones posteriores con `load_local()`, lo que evita re-vectorizar el corpus en cada ejecución. El mecanismo de actualización incremental se describe en la sección 3.3.5.

En la fase de consulta, el índice actúa como *retriever* mediante `as_retriever(search_kwargs={"k":10})` y recupera los 10 fragmentos más próximos al vector de consulta.

3.3.5. Gestión incremental

La vectorización del corpus con un modelo ejecutado localmente tiene un coste computacional considerable. Rehacer el índice desde cero cada vez que se añade un documento nuevo no resulta viable, por lo que se implementaron dos mecanismos: un registro persistente de documentos ya procesados y una actualización incremental del índice FAISS.

El fichero `procesados.txt` (disponible en la carpeta `data/` del repositorio descrito en el Anexo A) guarda los identificadores de todos los documentos que ya están indexados. Al ejecutar el script, el sistema lee su contenido línea por línea y crea un conjunto en memoria (`processed_files`). Antes de procesar cualquier fuente, comprueba si su identificador está ya en la lista. Para páginas web es la URL completa; para archivos PDF, es el nombre del fichero. Solo las fuentes que no aparecen en el conjunto entran en el *pipeline*; el resto se salta sin leerlo. Al terminar cada lote, los identificadores nuevos se añaden al fichero en modo *append*, sin modificar el contenido anterior, y el registro crece con cada ejecución.

Una limitación del diseño radica en que el identificador es el nombre del archivo y no un hash de su contenido. Si aparece una versión actualizada de un PDF con el mismo nombre, el sistema no lo detecta y no lo reindexa. Por esa razón, eliminar manualmente la línea correspondiente en `procesados.txt` es la única forma de forzar el reprocesamiento. Se asumió esta limitación porque, en el marco de este proyecto, el corpus permaneció estático durante toda la fase experimental.

En cuanto a la actualización del índice, la primera ejecución lo crea desde cero. Las siguientes lo cargan con `load_local()` y añaden los fragmentos nuevos con `add_documents()`, sin necesidad de reconstruirlo.

El procesamiento de los documentos se organiza en lotes y opera en dos niveles para controlar la memoria del servidor Ollama y evitar la pérdida de trabajo si el proceso se interrumpe. El primer nivel agrupa archivos en el buffer de la sesión y, al alcanzar 5 ficheros o 200 páginas, lo que suceda antes, se lanza el lote completo con segmentación, vectorización, actualización del índice y guardado en disco. Después el buffer se vacía y el registro se actualiza. El doble criterio existe porque algunos

documentos son muy largos y un límite basado únicamente en el número de ficheros no sería suficiente. Dentro de cada lote, los *chunks* tampoco se vectorizan todos a la vez, sino en grupos de cinco fragmentos, lo que reduce la carga sobre el servidor en cada petición y permite monitorizar el avance durante la indexación.

El guardado tras cada lote aporta robustez ante fallos. Si el proceso se interrumpe por un error de red, una excepción no controlada o cualquier otra causa, los lotes ya completados están guardados en disco. El registro `procesados.txt` evita que esos ficheros se vuelvan a procesar en la siguiente ejecución. Solo se pierden los fragmentos del lote activo en el momento del fallo, y ese número está limitado por `BATCH_SIZE=5` y el límite de 200 páginas.

3.4. Componente conversacional

Una vez construido el índice vectorial, el sistema entra en la fase de generación, en la que responde a las consultas del usuario en tiempo real. Esta sección describe el componente conversacional que se ocupa de esa fase. A diferencia del pipeline de ingesta, que se ejecuta de forma anticipada y produce un artefacto persistente en disco, el componente conversacional opera online y construye la respuesta en el momento en que llega cada pregunta.

El componente se organiza en tres partes. La primera describe la memoria conversacional y cómo se utiliza para reformular cada pregunta de forma independiente del historial antes de la búsqueda. La segunda define el comportamiento del modelo mediante un *prompt* de sistema con reglas explícitas. La tercera ensambla la cadena que une *retriever* y generador, y describe el modo de interacción con el usuario.

3.4.1. Memoria conversacional

En un sistema con memoria conversacional, las preguntas del usuario no suelen contener toda la información necesaria para interpretarse de forma aislada. Después de un primer intercambio, lo natural es continuar con referencias implícitas. Por ejemplo, si el usuario pregunta primero «¿qué rutas hay en la Sierra de Cazorla?» y, a continuación, «¿cuánto se tarda en hacerla?», la segunda pregunta solo tiene sentido en el contexto del intercambio anterior. Si el *retriever* procesa esa segunda pregunta sin modificarla, los fragmentos recuperados resultarán poco relevantes, ya que el vector de «¿cuánto se tarda en hacerla?» no contiene información sobre qué ruta concreta es «la».

Como solución a este problema, se utiliza un *history-aware* retriever, implementado con la función `create_history_aware_retriever` de `LangChain`. Su fun-

cionamiento se basa en una llamada previa al modelo de lenguaje antes de hacer la recuperación. El SLM recibe el historial de la conversación y la última pregunta del usuario, y devuelve una versión reformulada de la pregunta que pueda interpretarse sin acceso al historial. Siguiendo el ejemplo anterior, la pregunta «¿cuánto se tarda en hacerla?» se reformularía como «¿cuánto se tarda en hacer una ruta de la Sierra de Cazorla?», que ya contiene toda la información necesaria para la búsqueda vectorial.

La reformulación se controla mediante un *prompt* específico, `contextualize_q_system_prompt` (Anexo B.3), que define dos instrucciones para el modelo. La primera es transformar la pregunta para que sea independiente del historial. La segunda es no responder a la pregunta, únicamente reformularla. La segunda instrucción es crucial, ya que sin ella el modelo tiende a generar la respuesta en lugar de la consulta reformulada, lo que interrumpe el flujo del *pipeline*.

Cuando el historial está vacío, `create_history_aware_retriever` pasa la pregunta del usuario directamente al *retriever* sin llamar al modelo de lenguaje. La llamada adicional al SLM se hace solo cuando hay historial previo y la pregunta requiere reformulación.

Este historial reside en una lista de mensajes que se pasa en cada llamada al `rag_chain`. La lista contiene objetos `HumanMessage` y `AIMessage` de `LangChain`, alternados en el orden en que se produjeron. Tras cada interacción, la pregunta del usuario y la respuesta generada se añaden al final de la lista, y la versión actualizada se utiliza en la siguiente consulta.

Para evitar que el contexto crezca de forma indefinida, el historial está limitado a los diez mensajes más recientes (cinco intercambios completos). Una vez superado ese umbral, los mensajes más antiguos se descartan, como muestra la figura 3.3. Un historial más largo permite que el modelo mantenga referencias a preguntas y respuestas anteriores y produzca respuestas más relevantes en conversaciones extensas. Un historial más corto reduce el número de *tokens* que el SLM procesa en cada llamada, lo que disminuye la latencia y el consumo de memoria del servidor Ollama. El valor de diez se eligió como punto de partida para los primeros *tests* interactivos, suficiente para mantener una conversación con varios intercambios sin saturar el contexto del modelo. No se ha realizado un ajuste sistemático de este parámetro.

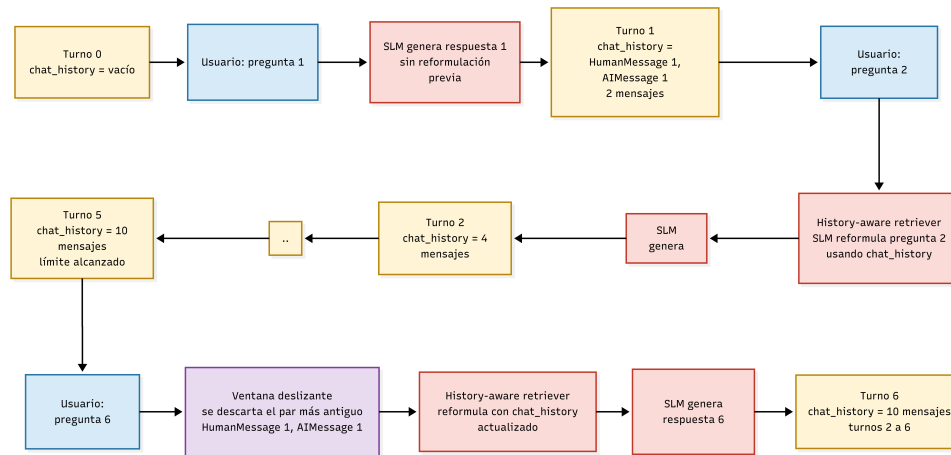


Figura 3.3. Control del historial con ventana deslizante.

La memoria conversacional no es persistente. Reside únicamente en la variable `chat_history` dentro del proceso de Python, y se pierde cuando el script termina. Como consecuencia, cada sesión de chat empieza desde cero, sin que el sistema conserve información de conversaciones anteriores. Esta limitación es coherente con el carácter de prototipo del sistema y se describe con más detalle en el capítulo 6.

En el modo test, el comportamiento es distinto. Dado que cada pregunta del Excel se evalúa de forma independiente, el historial se pasa siempre vacío en la llamada al `rag_chain`. Esto garantiza que la respuesta a cada pregunta no dependa de las preguntas anteriores del lote, lo que es necesario para que las métricas de evaluación sean comparables entre iteraciones.

3.4.2 *Prompt* del sistema

El *prompt* del sistema define el comportamiento del modelo durante la generación de la respuesta. En la arquitectura RAG, el *prompt* cumple dos funciones distintas. Por un lado, fija el rol del asistente y las restricciones sobre la información que puede utilizar. Por otro lado, actúa como contenedor del contexto recuperado, que se inserta en el `placeholder {context}` antes de pasar el conjunto al modelo.

El diseño del *prompt* en este sistema empezó con una versión inicial de carácter general, que se limitaba a indicar al modelo que debía responder sobre turismo y medio ambiente en Andalucía usando el contexto proporcionado. Esta versión, que se mantuvo sin cambios en las implementaciones v1, v2 y v2.1 del bot, producía resultados inconsistentes. El modelo a veces respondía con información que no estaba en los documentos recuperados, sobre todo cuando la pregunta trataba sobre aspectos geográficos generales (distancias entre municipios, ubicación de provincias). En otros casos, cuando el contexto correspondía a una zona distinta a la consultada por el usuario, el modelo presentaba esa información como si correspondiera a la zona preguntada.

A partir de estas observaciones, el *prompt* se reescribió de forma estructurada con cinco reglas, ordenadas de lo general a lo particular. La tabla 3.1 resume las dos versiones y la motivación de cada cambio. El texto completo de ambas versiones del *prompt* se recoge en el Anexo B.

Tabla 3.1. Evolución del *prompt* del sistema entre las versiones v2.1 y v2.2.

Versión	Contenido	Limitación
v2.1	Instrucción general. Responder basándose en el contexto, usar conocimiento general solo para preguntas geográficas.	Comportamiento inconsistente, mezcla de zonas geográficas, ausencia de manejo de peticiones de archivos descargables y formato variable.
v2.2	Cinco reglas estructuradas: fuentes de información, conocimiento general permitido, filtro geográfico, archivos descargables y formato de salida.	Comportamiento más predecible y trazable. Cada respuesta se puede asociar a una de las reglas.

La regla 1 establece qué información debe extraerse exclusivamente del contexto recuperado. Incluye los datos técnicos de las rutas (longitud, desnivel, duración, dificultad), la flora y fauna, los puntos de inicio y aparcamiento, y las restricciones o normativas. Si esa información no está disponible en los fragmentos recuperados, el modelo debe indicarlo de forma clara con una fórmula concreta, en lugar de inventarla o aproximarla.

La regla 2 define el conocimiento general que el modelo puede aportar. Cubre la geografía básica de Andalucía (provincias, ubicación de municipios, distancias entre localidades), el clima y la estacionalidad de la región, las recomendaciones genéricas de equipamiento, y la clasificación estándar de dificultad en senderismo. Esta regla soluciona un problema concreto observado en la versión anterior, en la que el modelo se negaba a contestar preguntas geográficas simples por no encontrarlas en el contexto, lo que deterioraba la experiencia del usuario.

La regla 3 introduce un filtro geográfico explícito. Cuando los fragmentos recuperados corresponden a una zona distinta a la consultada por el usuario, el modelo debe advertirlo antes de proporcionar la información. Esta regla responde a una limitación propia del *retriever*, que recupera los fragmentos más similares semánticamente sin garantizar coincidencia geográfica. Dado que el corpus cubre las ocho provincias, esta situación es frecuente.

La regla 4 trata consultas de archivos descargables (GPX, PDF, mapas). Como el sistema no tiene capacidad de enviar archivos al usuario, el modelo redirige al portal oficial de la Junta de Andalucía. Es el único caso de redirección al portal. En el resto de las consultas, el sistema debe responder directamente al usuario. Esta regla evita que el modelo prometa funcionalidades que no puede ejecutar.

La regla 5 fija el formato de salida. Indica el uso de negrita para nombres de rutas y conceptos clave, listas con guiones, y brevedad para no repetir información ya presente en el historial.

Al final del *prompt* se inserta el contexto recuperado mediante el **placeholder** `{context}`, que **LangChain** rellena automáticamente con el conjunto de los fragmentos devueltos por el *retriever*.

3.4.2. Cadena RAG

La cadena RAG conecta el *retriever* con el modelo generativo en un único flujo ejecutable. En **LangChain**, esta conexión se construye con dos funciones complementarias. La primera, `create_stuff_documents_chain`, recibe el modelo y el *prompt*, y produce una cadena que toma los fragmentos recuperados y los inserta en el **placeholder** `{context}` antes de pasar el *prompt* completo al modelo.

La segunda función, `create_retrieval_chain`, integra el flujo completo. Recibe el **history-aware** retriever y la cadena de documentos, y produce el objeto `rag_chain` que el sistema invoca en cada consulta. Al recibir la pregunta del usuario y el historial del chat, la cadena ejecuta los pasos en orden, como muestra la figura 3.3. Primero llama al **history-aware** retriever, que reformula la pregunta si es necesario y devuelve los fragmentos relevantes. A continuación, pasa esos fragmentos junto con la pregunta original y el historial a la cadena de documentos, que construye el *prompt* final y lo envía al modelo. El resultado contiene la respuesta generada y los fragmentos utilizados como contexto. La figura 3.4 muestra el flujo completo de la cadena.

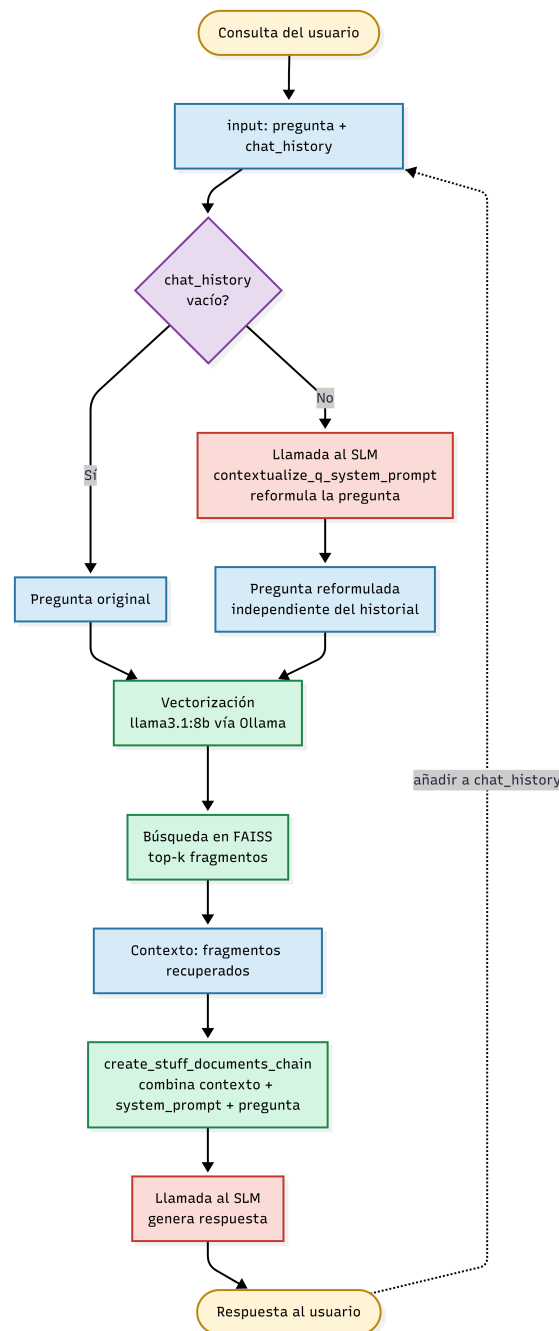


Figura 3.4. Flujo de la cadena RAG en la fase de generación.

El sistema registra para cada consulta la respuesta generada y los fragmentos de contexto utilizados. Esta información se utiliza en el modo chat para mostrar las fuentes al usuario, y en la fase de evaluación (capítulo 4) para calcular las métricas RAGAS, que requieren saber qué fragmentos contribuyeron a la respuesta.

El sistema ofrece un modo interactivo de chat que permite al usuario formular consultas en lenguaje natural. La interfaz es un bucle por consola que solicita la pregunta, ejecuta la cadena RAG y muestra la respuesta generada. Para fines de

desarrollo y verificación, el sistema imprime también los fragmentos utilizados como contexto y el documento del que proceden. Esta información permite comprobar manualmente que la respuesta se apoya en fuentes relevantes, aunque su formato en la consola no está orientado al uso final del usuario.

El bucle continúa hasta que el usuario introduce «exit» o «stop». Entre cada consulta, el sistema mantiene el historial actualizado, de modo que el usuario puede formular preguntas que dependen del contexto del intercambio anterior, como las descritas en 3.4.1.

3.5. Entorno de ejecución

El sistema se desplegó en un servidor proporcionado por el grupo de investigación HUMAINS (*human-centred artificial intelligence methods, applications, and software*; TIC-280) de la Universidad de Córdoba. La máquina cuenta con un procesador AMD Ryzen 9 7950X de 16 núcleos físicos y 32 hilos, 64 GB de memoria RAM y una GPU NVIDIA GeForce RTX 3060 Ti con 8 GB de memoria dedicada (VRAM). El sistema operativo es Ubuntu 24.04.4 LTS sobre kernel 6.8, con CUDA 12.2 disponible para la aceleración por GPU. Esta configuración corresponde a un servidor de propósito general, sin el hardware especializado que se asocia habitualmente al entrenamiento o inferencia de modelos de lenguaje a gran escala, lo cual resulta coherente con el planteamiento del trabajo y con la motivación de utilizar modelos de lenguaje pequeños adecuados para ejecutarse sobre infraestructura convencional (sección 1.1).

Como modelo de lenguaje se utiliza llama3.1:8b, que se ejecuta sobre la GPU a través de Ollama dentro de los 8 GB de VRAM disponibles. La elección de este modelo responde a dos motivos. Por un lado, es uno de los modelos abiertos del rango de 7 a 9 mil millones de parámetros que incluye el español entre sus lenguas oficialmente soportadas (Meta AI, 2024), lo que es importante porque el corpus y las consultas se formulan en español. Por otro, su tamaño se ajusta a la VRAM disponible. El mismo modelo se utiliza tanto para la generación de respuestas como para el cálculo de los *embeddings* del corpus y de las consultas. El uso de un único modelo para ambas tareas simplifica la configuración del entorno y reduce las dependencias, aunque condiciona el rendimiento de la fase de recuperación. Este aspecto se describe en el capítulo 6 al discutir las limitaciones del sistema.

El *pipeline* se implementó en Python 3.12 dentro de un entorno virtual aislado. La integración entre los distintos componentes se apoya en el ecosistema LangChain (versión 0.3), que proporciona la abstracción de cadenas conversacionales con memoria, el conector con Ollama (LangChain-ollama) y el *wrapper* sobre el índice vectorial. El

índice vectorial se construye con FAISS en su variante sobre CPU (`faiss-cpu`). La variante con aceleración por GPU está justificada para corpus del orden de millones de vectores, mientras que el fichero de vectores resultante en la configuración baseline ocupa 186 MB en disco (sección 3.3.4). Además, la GPU disponible está dedicada a la inferencia del modelo de lenguaje.

La limitación más relevante del entorno se refiere al tamaño de la VRAM disponible. Con 8 GB, la GPU es suficiente para alojar el modelo utilizado en este trabajo, pero impone un techo tanto al tamaño del modelo como al tamaño máximo de contexto que pueden ser utilizados sin que parte del cómputo se desplace a CPU. Esta restricción condiciona algunas decisiones de diseño del capítulo 5 y se retoma en la sección 6.2.

Marco de evaluación

4.1. *Framework* de evaluación

La sección 2.7 presentó cuatro *frameworks* de evaluación automática de sistemas RAG: TruLens, DeepEval, ARES y RAGAS. Para este trabajo se eligió RAGAS por dos motivos. El primero es práctico. RAGAS es una librería de Python que se instala con un único comando y que se invoca como una función más sobre los datos, sin necesidad de levantar un servicio de monitorización, como hace TruLens ni de configurar un *pipeline* de integración continua como suele requerir DeepEval. El segundo motivo se basa en el propósito del trabajo. TruLens y DeepEval están pensados sobre todo para monitorizar sistemas RAG en producción a lo largo del tiempo, mientras que RAGAS encaja mejor en un escenario de evaluación experimental, donde el objetivo es obtener puntuaciones comparables entre varias configuraciones probadas de forma controlada. ARES quedó descartado desde el principio porque necesita una fase previa de entrenamiento de clasificadores sobre unas 150 muestras anotadas (Saad-Falcon et al., 2023), un coste que no estaba justificado en el alcance del trabajo.

El LLM-juez se configura en el módulo `evaluate_ragas.py`. RAGAS no requiere un modelo concreto y recibe el LLM ya inicializado, envuelto en la clase `LangChainLLMWrapper`. En este trabajo el juez es el mismo `ChatOllama` con el modelo `llama3.1:8b` que ya usa el bot para generar respuestas. Los *embeddings* de

evaluación se obtienen del mismo `OllamaEmbeddings` que utiliza el *retriever*. De esta forma toda la evaluación se ejecuta en local, sobre el servidor de Ollama del proyecto, sin necesidad de añadir nuevas dependencias ni de llamar a APIs externas. Esta decisión conlleva consecuencias, ya que el modelo evaluado y el modelo juez son el mismo. Este efecto se conoce como *self-preference bias* (Panickssery et al., 2024) y se describe en 6.2.

Cada evaluación recibe como entrada un objeto `SingleTurnSample` con tres campos: `user_input` (la pregunta del usuario), `response` (la respuesta generada por el bot) y `retrieved_contexts` (los fragmentos recuperados del índice vectorial). Cada `SingleTurnSample` representa una sola interacción pregunta-contexto-respuesta. RAGAS devuelve una puntuación por muestra y por métrica, y esos valores se agregan después sobre las repeticiones de cada escenario, como se explica en 4.5.

4.2. Métricas

Entre las métricas que ofrece RAGAS se han seleccionado tres, una por cada dimensión de la tríada de evaluación. *Faithfulness* mide la fidelidad de la respuesta al contexto recuperado. *Response Relevancy* mide si la respuesta aborda lo que pregunta el usuario. *Context Precision* en su versión sin referencia mide la calidad del ranking de los fragmentos recuperados. Las tres devuelven un valor entre 0 y 1, donde valores cercanos a uno indican mejor comportamiento del sistema. *Faithfulness* y *Response Relevancy* siguen la formulación de la propuesta original de RAGAS (Es et al., 2023). *Context Precision* se incorporó después y su definición se basa en la documentación del *framework* (Ragas, 2026).

Faithfulness

Faithfulness mide qué porcentaje de las afirmaciones contenidas en la respuesta se apoya en el contexto recuperado. El cálculo tiene dos pasos. Primero, el LLM-juez descompone la respuesta en un conjunto de afirmaciones atómicas $S(a_s(q))$. Después, para cada afirmación $s_i \in S$ el juez comprueba si esa afirmación está respaldada por el contexto $c(q)$ y responde sí o no. La puntuación final se calcula como:

$$F = \frac{|V|}{|S|} \quad (4.1)$$

donde $|V|$ es el número de afirmaciones respaldadas por el contexto y $|S|$ es el número total de afirmaciones extraídas. Una puntuación baja indica que parte del contenido de la respuesta no se apoya en los fragmentos recuperados.

Response Relevancy

Response Relevancy mide si la respuesta aborda lo que se ha preguntado, sin evaluar si la información es verdadera o no. El cálculo parte de la respuesta generada y pide al LLM-juez que genere n preguntas hipotéticas q_i para las que esa respuesta sería adecuada. Después se obtiene el *embedding* de la pregunta original q y de cada q_i , y se calcula la similitud del coseno entre ellos. La puntuación final es la media de las n similitudes:

$$\text{AR} = \frac{1}{n} \sum_{i=1}^n \text{sim}(q, q_i) \quad (4.2)$$

Los valores bajos indican que la respuesta es incompleta o introduce información que no está conectada con la pregunta, porque en esos casos las preguntas que el juez reconstruye a partir de la respuesta se alejan de la pregunta original. Una respuesta puede apoyarse en el contexto y aun así no responder a lo preguntado, por eso *Response Relevancy* aporta información distinta a *Faithfulness*.

Context Precision (sin referencia)

En este trabajo se utiliza *LLMContextPrecisionWithoutReference*, denominada también *Context Utilization* en la documentación de RAGAS. La métrica mide la calidad del *ranking* de los fragmentos recuperados y premia que los fragmentos relevantes ocupen las primeras posiciones. Para cada fragmento de la lista `retrieved_contexts` el LLM-juez lo compara con la respuesta generada y decide si es relevante, lo que produce un indicador binario $v_k \in \{0, 1\}$ en la posición k . La puntuación se calcula como:

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \cdot v_k)}{\text{número total de fragmentos relevantes en el top } K} \quad (4.3)$$

donde *Precision@k* es la proporción de fragmentos relevantes entre las primeras k posiciones y K es el número total de fragmentos recuperados. La fórmula multiplica *Precision@k* por el indicador v_k , de manera que solo suman las posiciones que contienen un fragmento relevante. Como *Precision@k* depende del orden de los primeros k fragmentos, dos configuraciones que recuperen los mismos documentos pueden obtener puntuaciones distintas si los presentan en distinto orden. Esto la convierte en una métrica útil para comparar configuraciones del *retriever*.

Las tres métricas se aplican por separado sobre cada `SingleTurnSample` y devuelven un vector de tres valores por muestra. Cada una se centra en un aspecto distinto del *pipeline*. Una caída en *Context Precision* apunta al *retriever*, una caída en *Faithfulness* sugiere problemas en la generación a pesar de contar con un contexto adecuado, y una caída en *Response Relevancy* indica que la respuesta no se ajusta a

la pregunta. El protocolo experimental que se describe en la sección 4.3 aprovecha esta separación al comparar configuraciones que modifican un solo componente del *pipeline*.

4.3. Protocolo experimental

El conjunto de evaluación está formado por 60 preguntas en español, seleccionadas de manera aleatoria de un corpus de unas 400 preguntas elaborado en el contexto del proyecto NATURCOR-IA (sección 1.2). Evaluar las 400 preguntas en cada una de las doce configuraciones, con diez repeticiones por pregunta y con la evaluación RAGAS aplicada sobre cada respuesta, no era viable en el entorno local de ejecución descrito en la sección 3.5, por lo que el tamaño de la muestra se fijó en 60 como equilibrio entre la representatividad del conjunto y el tiempo total de cómputo del estudio. Las preguntas reflejan consultas reales que un usuario podría dirigir a un asistente sobre rutas y espacios naturales de Andalucía, formuladas con lenguaje coloquial y escenarios concretos como salidas con niños, accesibilidad o recomendaciones cerca de un municipio concreto. La selección se hizo una sola vez y se mantuvo fija en todos los experimentos, de manera que las diferencias observadas entre configuraciones no dependieran de cambios en la muestra. El conjunto no incluye respuestas de referencia, ya que las tres métricas RAGAS utilizadas en este trabajo son *reference-free* y evalúan cada respuesta a partir de la pregunta y del contexto recuperado (sección 4.1).

Cada configuración se evalúa sobre esas 60 preguntas con 10 repeticiones por pregunta. El modelo no produce respuestas idénticas ante la misma entrada, así que una sola ejecución no es suficiente para describir el comportamiento medio de una configuración. El valor de 10 se fijó como punto razonable entre estabilidad de las medias y coste computacional del experimento, ya que cada pregunta supone una llamada completa al *pipeline* RAG. El resultado son 600 puntuaciones por configuración y por métrica.

Para aislar el efecto de cada variable se fija el resto del entorno experimental. Todos los experimentos parten del sistema de referencia descrito en la sección 5.1, que define la configuración base sobre la que se introduce una única variación por experimento. El parámetro k del *retriever* no tiene un valor fijo, ya que se ajusta cuando es necesario para que el volumen total de contexto inyectado en el *prompt* se mantenga alrededor de 10.000 caracteres (sección 3.3.2).

Para cada ejecución, el protocolo registra la pregunta, la iteración, la respuesta generada, los fragmentos de contexto recuperados, las fuentes documentales y el tiempo de respuesta del *pipeline*. La pregunta, la respuesta y los fragmentos son la

entrada de las métricas RAGAS descritas en 4.2. El tiempo y las fuentes se utilizan para analizar el coste de cada configuración e identificar de dónde proceden las respuestas.

4.4. Implementación

El protocolo se implementa en el *script* `evaluate_ragas.py`, del que existe una variante por cada experimento del capítulo 5 con las modificaciones que la configuración requiere, ya sea en el *retriever*, la base de datos vectorial o los parámetros de búsqueda. El *script* tiene dos modos de uso. El primero recibe un Excel con respuestas ya generadas y solo calcula las métricas RAGAS sobre él. El segundo ejecuta el *pipeline* RAG sobre todo el conjunto de evaluación y evalúa cada respuesta después de generarla. Todos los experimentos se realizaron con el segundo modo, ya que el formato de almacenamiento del contexto en el Excel del primer modo une los fragmentos en una única cadena, lo que impide a *Context Precision* evaluar el ranking de los fragmentos individuales.

En cada iteración, la pregunta se envía a la cadena `rag_chain` con el historial de chat vacío, de modo que la respuesta no dependa de preguntas anteriores del lote (sección 3.4.1). Una vez generada la respuesta, el *script* construye un objeto `SingleTurnSample` y lo pasa a cada una de las tres métricas RAGAS. El resultado se añade como una fila en un Excel de salida con las siguientes columnas: pregunta, número de iteración, tiempo de generación, estimación de *tokens* de entrada y salida, respuesta, fuentes documentales, contexto utilizado, las tres métricas RAGAS y el tiempo dedicado a la evaluación. El tiempo de generación corresponde al *pipeline* RAG completo, desde la recepción de la pregunta hasta la respuesta del modelo, sin incluir la evaluación posterior. La estimación de *tokens* se calcula como el número de palabras separadas por espacios multiplicado por 1,3. Esta estimación no se utiliza para el cálculo de las métricas y se registra como información complementaria por las razones señaladas en la sección 3.3.2 al elegir caracteres como unidad de segmentación. Las iteraciones que terminan con error se añaden a la tabla de salida con los campos de respuesta y métricas vacíos.

El sistema base sobre el que se ejecutan los experimentos corresponde a la versión 2.2 del bot descrita en el capítulo 3. El índice vectorial se reutiliza entre experimentos, salvo en aquellos en los que la configuración exige reconstruirlo.

4.5. Análisis estadístico

El protocolo descrito en la sección 4.3 produce, para cada configuración, una matriz de 600 puntuaciones por métrica (60 preguntas, 10 iteraciones). Antes de comparar configuraciones entre sí, los datos se agregan a nivel de pregunta tomando la mediana de las 10 iteraciones, lo que reduce cada configuración a un vector de 60 valores por métrica. El análisis se aplica a las doce configuraciones evaluadas en el capítulo 5. Se elige la mediana en lugar de la media porque las puntuaciones RAGAS están limitadas en $[0, 1]$ y tienen asimetría, como se discute más abajo, lo que hace a la mediana un estimador del comportamiento más robusto frente a iteraciones atípicas. Las medias se calculan como complemento descriptivo en las tablas de resultados.

Para decidir entre tests paramétricos y no paramétricos se aplicó el test de Shapiro-Wilk, que evalúa si la distribución de un conjunto de valores se aproxima a una distribución normal: un p -value bajo indica que la hipótesis de normalidad se rechaza. Se aplicó a las medianas por pregunta de cada configuración y métrica, junto con los coeficientes de asimetría y curtosis como diagnóstico complementario. La tabla 4.1 resume los resultados. *Response Relevancy* y *Context Precision* rechazan la normalidad en las doce configuraciones, y *Faithfulness* la rechaza en siete de ellas. Estos rechazos son consistentes con la naturaleza de las métricas, ya que los valores están limitados a $[0, 1]$, lo que impide colas simétricas como las de la distribución normal y obliga a que, cuando la métrica se concentra cerca de uno de los extremos del intervalo, la distribución sea asimétrica. Esto es especialmente visible en *Context Precision*, cuyas medianas están por encima de 0,9 en la mayoría de las configuraciones. Por tanto, las tres métricas se analizan con tests no paramétricos.

Tabla 4.1. Resultados del test de normalidad de Shapiro-Wilk para las métricas evaluadas.

Métrica	Rechazo ($p < 0,05$)	Asimetría (rango)	Curtosis (rango)
<i>Faithfulness</i>	7/12	de -0,09 a +0,52	de -1,20 a +1,55
<i>Response Relevancy</i>	12/12	de -3,39 a -1,18	de +2,61 a +17,77
<i>Context Precision</i>	12/12	de -3,42 a -1,22	de +0,16 a +12,30

El esquema general sigue las recomendaciones de Demšar (2006) para la comparación de sistemas sobre múltiples conjuntos de datos. Se distinguen dos niveles de análisis: un test global por grupo de experimentos, que decide si existe alguna diferencia significativa entre las configuraciones del grupo, y tests por pares, que identifican entre qué configuraciones concretas se producen las diferencias.

Para cada grupo de experimentos con tres o más configuraciones se aplica el test de Friedman, que opera sobre los rangos de las puntuaciones dentro de cada pregunta. Dadas k configuraciones evaluadas sobre N preguntas, el estadístico de Friedman se define como:

$$\chi_F^2 = \frac{12N}{k(k+1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k+1)^2}{4} \right] \quad (4.4)$$

donde R_j es el rango medio de la configuración j sobre las N preguntas. El test parte de la suposición de que todas las configuraciones del grupo son equivalentes. Un p -value inferior a 0,05 indica que esta suposición no se sostiene y que al menos una configuración se comporta de forma distinta, lo que justifica pasar a las comparaciones por pares.

Cuando el test de Friedman detecta diferencias, o cuando el grupo contiene solo dos configuraciones, se aplica el test de Wilcoxon para pares relacionados. El test calcula las diferencias $d_i = x_i - y_i$ entre los valores pareados por pregunta, descarta las diferencias nulas, ordena las restantes por valor absoluto y asigna rangos. El estadístico del test se calcula como:

$$W = \min(T^+, T^-), \quad \text{con } T^+ = \sum_{d_i > 0} R_i \text{ y } T^- = \sum_{d_i < 0} R_i \quad (4.5)$$

Un p -value bajo indica que una de las dos configuraciones obtiene puntuaciones consistentemente mayores que la otra, y que esta diferencia no se debe al azar. En este trabajo, Wilcoxon se utiliza con dos propósitos: comparar cada configuración con el sistema de referencia CS-1000 (sección 5.1) y comparar configuraciones del mismo grupo entre sí.

La aplicación de varios tests Wilcoxon dentro de un mismo grupo de experimentos aumenta la probabilidad de obtener falsos positivos. Para corregir este efecto, se aplica la corrección de Holm-Bonferroni dentro de cada grupo. Si en el grupo se han realizado m comparaciones y p_i es el i -ésimo p -value en orden ascendente, el p -value corregido es

$$p_i^{(\text{Holm})} = \min \left[(m - i + 1) \cdot p_i, 1 \right] \quad (4.6)$$

El p -value más pequeño se multiplica por m , el segundo por $m-1$, el tercero por $m-2$, y el último queda sin modificar. De este modo, los resultados más extremos pasan un filtro más estricto. La corrección se realiza por grupo y no sobre el conjunto completo de los 12 experimentos, dado que cada grupo responde a una pregunta de investigación distinta y las comparaciones entre grupos no son intercambiables.

Un p -value significativo indica que la diferencia entre dos configuraciones es difícilmente atribuible al azar, pero no informa de la magnitud de esa diferencia. Para complementar el p -value con una medida de la magnitud del efecto, se calcula el coeficiente *rank-biserial* r a partir de los rangos de Wilcoxon:

$$r = \frac{T^+ - T^-}{n(n+1)/2} \quad (4.7)$$

donde n es el número de pares con diferencia no nula. El signo indica la dirección del efecto y su magnitud se interpreta según los umbrales de Cohen (1988): efecto pequeño a partir de $|r| \geq 0,1$, mediano a partir de $|r| \geq 0,3$ y grande a partir de $|r| \geq 0,5$. Por debajo de 0,1 el efecto se considera insignificante con independencia de la significancia estadística.

Para cada comparación se reportan la mediana de cada configuración, la diferencia entre medianas, el p -value original, el p -value tras la corrección de Holm, el coeficiente r y su interpretación cualitativa. En la discusión del capítulo 5, los resultados que obtienen significancia tras la corrección de Holm ($p < 0,05$) con un efecto al menos mediano ($|r| \geq 0,3$) se interpretan como diferencias firmes respecto al sistema de referencia. Los resultados con p próximos al umbral (entre 0,05 y 0,10) y efecto al menos mediano se interpretan como tendencias que no alcanzan la significancia formal, pero indican un comportamiento sistemático. La ausencia de diferencias significativas se interpreta como indicio de que la configuración evaluada no aporta una mejora medible respecto al sistema de referencia. Todos los cálculos descritos en esta sección se implementan en el *script statistical_analysis.py*, que produce un Excel con cinco hojas: estadísticas descriptivas, test de normalidad, test de Friedman por grupo, comparaciones Wilcoxon frente al sistema de referencia y comparaciones Wilcoxon intragrupo.

Experimentación y resultados

La tabla 5.1 resume todas las configuraciones evaluadas en este capítulo y los parámetros que se mantienen fijos en cada una. El sistema de referencia (CS-1000) actúa como baseline frente al que se comparan el resto de las configuraciones. Cada experimento de las secciones 5.2 a 5.6 modifica un único aspecto del sistema y mantiene el resto idéntico al baseline.

Tabla 5.1. Configuraciones evaluadas y parámetros fijos del experimento.

Grupo	Configuración	Descripción
Sistema de referencia		
<i>Baseline</i>	<i>CS-1000</i>	<code>chunk_size</code> = 1.000 caracteres, <code>chunk_overlap</code> = 200, $k = 10$. Modelo único llama3.1:8b para generación y <i>embeddings</i> .
Configuraciones evaluadas		
Tamaño de <i>chunk</i>	<i>CS-500</i>	<code>chunk_size</code> = 500, <code>chunk_overlap</code> = 100, $k = 20$
	<i>CS-2000</i>	<code>chunk_size</code> = 2.000, <code>chunk_overlap</code> = 400, $k = 5$
<i>Retriever</i> híbrido	<i>H-50/50</i>	Búsqueda densa + BM25 combinadas con <i>Reciprocal Rank Fusion</i> . Peso semántico / BM25 = 0,5 / 0,5
	<i>H-30/70</i>	Peso semántico / BM25 = 0,3 / 0,7
	<i>H-70/30</i>	Peso semántico / BM25 = 0,7 / 0,3
<i>Multi-Query</i>	<i>MQ-B4x3</i>	4 subconsultas, $k = 3$ fragmentos por subconsulta
	<i>MQ-B3x4</i>	3 subconsultas, $k = 4$ fragmentos por subconsulta
<i>Rewrite-Retrieve-Read</i>	<i>RRR-AD-300</i>	<i>Prompt</i> de adaptación al vocabulario del corpus; primeros 300 caracteres de los 6 fragmentos iniciales como contexto
	<i>RRR-AD</i>	<i>Prompt</i> de adaptación al vocabulario; 10 fragmentos iniciales completos
	<i>RRR-GE</i>	<i>Prompt</i> orientado a precisar la geografía; 10 fragmentos iniciales completos
Metadatos	<i>META-B</i>	Inyección de metadatos estructurales en el contexto del <i>prompt</i>
Parámetros fijos en todas las configuraciones		
Modelo de lenguaje	llama3.1:8b	Vía Ollama. <i>temperature</i> = 0,8 (valor por defecto de Ollama). Usado para generación, <i>embeddings</i> y como LLM-juez de RAGAS
Índice vectorial	<i>FAISS</i> <i>IndexFlatL2</i>	Búsqueda exacta con distancia L2 (euclídea). Dimensión del vector = 4.096
Memoria conversacional	<i>10 mensajes</i>	Equivalente a 5 turnos. En evaluación el <code>chat_history</code> se pasa siempre vacío
Métricas RAGAS	<i>Reference-free</i>	<i>Faithfulness</i> , <i>Response Relevancy</i> y <i>Context Precision</i>
Protocolo experimental	60×10	60 preguntas con 10 iteraciones cada una. Agregación por pregunta mediante mediana de las 10 iteraciones

De las técnicas de RAG Avanzado se prueban tres: *retriever* híbrido (sección 5.3), *Multi-Query* (sección 5.4) y *Rewrite-Retrieve-Read* (sección 5.5). El *reranking* queda fuera del alcance del trabajo porque requiere un modelo adicional especializado en evaluar la relevancia (típicamente un *cross-encoder* como BGE-reranker), lo que rompería el criterio de utilizar un único modelo local descrito en la sección 3.5. *Self-RAG* también se descarta, ya que requiere un modelo entrenado expresamente con *reflection tokens* (Asai et al., 2023). Además del estudio de las técnicas de RAG Avanzado, el capítulo incluye experimentos sobre parámetros del *pipeline baseline*, concretamente el tamaño de *chunk* (sección 5.2) y la inyección de metadatos (sección 5.6).

5.1. Sistema de referencia

El sistema de referencia fija todos los parámetros del pipeline RAG en un valor de partida razonable y se utiliza como punto de comparación común en los tests estadísticos descritos en la sección 4.5. Adopta la configuración descrita en los capítulos 3 y 4 con un tamaño de *chunk* de 1.000 caracteres y un solapamiento de 200, valores justificados en la sección 3.3.2 como configuración inicial razonable a partir de la documentación de *LangChain* y de la literatura sobre RAG (Gao et al., 2024; Barnett et al., 2024). El *retriever* devuelve los $k = 10$ fragmentos más similares a la consulta mediante búsqueda densa con FAISS, lo que produce un contexto inyectado en el *prompt* de alrededor de 10.000 caracteres. La generación y el cálculo de *embeddings* se realizan con llama3.1:8b a través de Ollama, y la evaluación se realiza con las tres métricas RAGAS y el protocolo de 60 preguntas con 10 iteraciones descrito en la sección 4.3. Esta configuración se identifica como CS-1000 en el resto del capítulo.

La tabla 5.2 recoge las medidas descriptivas de las tres métricas RAGAS y el tiempo de respuesta del *pipeline* para CS-1000. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación, en coherencia con la agregación utilizada en los tests estadísticos. Para cada métrica se reportan la media, la mediana y la desviación estándar. El tiempo de respuesta muestra una desviación estándar muy alta y una media muy superior a la mediana, lo que indica la presencia de valores atípicos (*outliers*) que distorsionan la media.

Tabla 5.2. Estadísticas descriptivas del sistema de referencia CS-1000.

Métrica	Media	Mediana	Desv. estándar
<i>Faithfulness</i>	0,349	0,333	0,248
<i>Response Relevancy</i>	0,607	0,621	0,129
<i>Context Precision</i>	0,846	0,991	0,244
Tiempo RAG (s)	16,700	6,130	80,620

Las tres métricas están en niveles muy distintos. La mediana de *Context Precision* queda cerca del extremo superior del intervalo, mientras que *Faithfulness* se ubica en la parte baja del rango y *Response Relevancy* en una zona intermedia. La discusión detallada de estos niveles y de su relación con las características del sistema se retoma a lo largo del capítulo, a medida que cada experimento aporta nuevos puntos de comparación.

5.2. Tamaño de *chunk*

El primer experimento varía el tamaño de los fragmentos en los que se divide el corpus antes de indexarlo. Como se discute en la sección 3.3.2, el tamaño de *chunk* influye en el equilibrio entre la especificidad del vector y la cantidad de contexto que cada fragmento conserva, sin que exista un valor óptimo universal. El objetivo del experimento es medir el efecto de este parámetro sobre las métricas de calidad del sistema, manteniendo constantes el resto de los componentes del *pipeline*. Para que la comparación sea justa, el volumen total de contexto inyectado en el *prompt* se mantiene fijo alrededor de 10.000 caracteres ajustando el número de fragmentos recuperados k a cada tamaño de *chunk*. Se evalúan tres configuraciones:

- CS-500: *chunk* de 500 caracteres; solapamiento de 100; $k = 20$
- CS-1000: sistema de referencia; *chunk* de 1.000; solapamiento de 200; $k = 10$
- CS-2000: *chunk* de 2.000; solapamiento de 400; $k = 5$.

La tabla 5.3 recoge las medidas descriptivas de las tres métricas RAGAS y del tiempo de respuesta para cada configuración. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación.

Tabla 5.3. Estadísticas descriptivas del experimento de tamaño de *chunk*.

Métrica	Estadístico	CS-500	CS-1000	CS-2000
<i>Faithfulness</i>	Media	0,437	0,349	0,362
<i>Faithfulness</i>	Mediana	0,431	0,333	0,387
<i>Faithfulness</i>	Desv. estándar	0,268	0,248	0,231
<i>Response Relevancy</i>	Media	0,604	0,607	0,616
<i>Response Relevancy</i>	Mediana	0,630	0,621	0,624
<i>Response Relevancy</i>	Desv. estándar	0,108	0,129	0,119
<i>Context Precision</i>	Media	0,937	0,846	0,722
<i>Context Precision</i>	Mediana	1,000	0,991	0,845
<i>Context Precision</i>	Desv. estándar	0,161	0,244	0,332
Tiempo RAG (s)	Media	6,090	16,700	11,000
Tiempo RAG (s)	Mediana	6,210	6,130	5,690
Tiempo RAG (s)	Desv. estándar	1,220	80,620	40,430

La métrica que más varía entre configuraciones es *Context Precision*. La mediana pasa de 1 en CS-500 a 0,845 en CS-2000, con CS-1000 entre ambas. *Faithfulness* muestra una tendencia parecida, con CS-500 por encima del resto, aunque la diferencia es menor. *Response Relevancy* se mantiene prácticamente igual en las tres configuraciones. La mediana del tiempo de respuesta varía poco entre configuraciones, alrededor de seis segundos por consulta, lo que indica que el tamaño de *chunk* no afecta de forma notable al coste del *pipeline* cuando el volumen total de contexto se mantiene constante.

El test global de Friedman aplicado al grupo confirma estas observaciones. Para *Context Precision* detecta diferencias claras entre las tres configuraciones ($\chi^2 = 26,85$, $p < 0,001$), mientras que para *Faithfulness* y *Response Relevancy* los p-values son altos ($p = 0,312$ y $p = 0,514$), lo que indica que las tres configuraciones se comportan de forma equivalente en estas métricas. Las comparaciones por pares con el sistema de referencia confirman dónde se concentran las diferencias. En *Context Precision*, CS-500 mejora a CS-1000 con $p = 0,013$ tras la corrección de Holm y un efecto grande ($r = 0,51$), y CS-2000 lo empeora con $p = 0,013$ y un efecto mediano ($r = -0,44$). En *Faithfulness*, CS-500 obtiene una mejora cercana a la significancia ($p = 0,056$ tras Holm, $r = 0,33$), suficiente para considerarla una tendencia según el criterio establecido en la sección 4.5 pero sin alcanzar el umbral formal. CS-2000 no muestra diferencias con el sistema de referencia en esta métrica. En *Response Relevancy* ninguna configuración tiene diferencias significativas de CS-1000.

Las comparaciones por pares dentro del grupo confirman el patrón en *Context Precision*. CS-500 supera a CS-2000 con un efecto grande ($r = 0,81$, $p < 0,001$ tras Holm), CS-500 supera a CS-1000 con un efecto grande ($r = 0,51$, $p = 0,013$) y CS-1000 supera a CS-2000 con un efecto mediano ($r = 0,44$, $p = 0,013$). Las tres comparaciones por pares en esta métrica son significativas y muestran una tendencia consistente: cuanto menor es el tamaño de *chunk*, mayor es la precisión del contexto recuperado. En *Faithfulness* y *Response Relevancy* ninguna de las comparaciones intragrupo alcanza significancia tras la corrección.

El resultado más claro del experimento es que la reducción del tamaño de *chunk* de 1.000 a 500 caracteres mejora de forma significativa la precisión con la que el *retriever* ordena los fragmentos recuperados, mientras que aumentarlo a 2.000 la deteriora. La tendencia en *Faithfulness* apunta en la misma dirección, aunque sin alcanzar significancia formal. La estabilidad de *Response Relevancy* y del tiempo de respuesta entre configuraciones sugiere que el efecto observado se localiza en la fase de recuperación y no afecta de forma apreciable a la fase de generación ni al coste del *pipeline*. Estos resultados se discuten junto con el resto de experimentos en la sección 5.7.

5.3. *Retriever* híbrido

El segundo experimento evalúa un esquema de recuperación híbrida que combina la búsqueda densa basada en *embeddings* con la búsqueda léxica basada en BM25, siguiendo el enfoque descrito en la sección 2.3. La hipótesis que motiva el experimento es que la combinación de las dos señales pueda mejorar la calidad del contexto recuperado en los casos en los que la coincidencia exacta de términos resulta relevante, sin perder las ventajas de la búsqueda semántica frente a las reformulaciones de la consulta. La implementación utiliza *EnsembleRetriever* de **LangChain**, que combina los dos *retrievers* mediante *Reciprocal Rank Fusion*, un método que asigna a cada documento una puntuación basada en su posición en cada ranking individual y los combina. En las tres configuraciones evaluadas el *retriever* semántico y el *retriever* BM25 devuelven cinco fragmentos cada uno, y el ensemble produce los diez fragmentos finales que se inyectan en el *prompt*, manteniendo el volumen de contexto de 10.000 caracteres. La figura 5.1 ilustra el mecanismo.

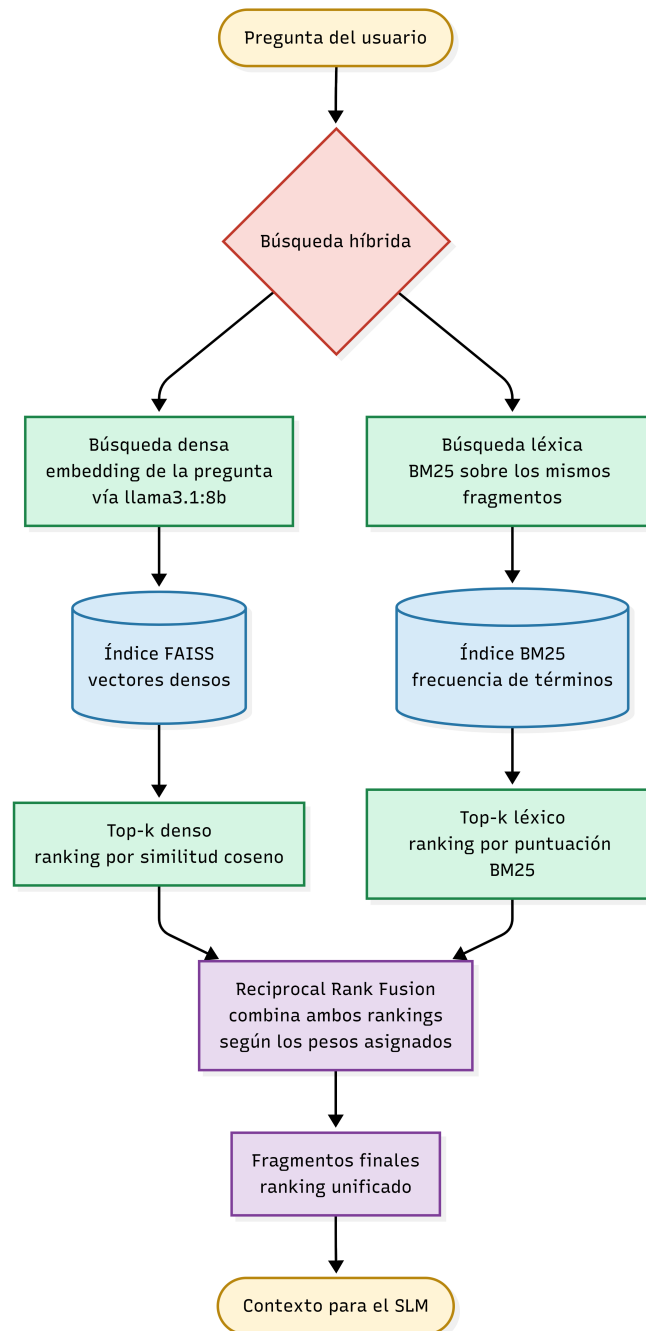


Figura 5.1. Mecanismo del *retriever* híbrido.

Lo que varía entre las tres configuraciones es el peso asignado a cada *retriever* dentro del ensemble. El primer valor del nombre corresponde al peso de la búsqueda semántica y el segundo al de BM25. Las tres configuraciones evaluadas son:

- H-50/50: pesos iguales (0,5 semántica; 0,5 BM25)
- H-30/70: prioriza la búsqueda léxica (0,3 semántica; 0,7 BM25)
- H-70/30: prioriza la búsqueda semántica (0,7 semántica; 0,3 BM25).

La tabla 5.4 recoge las medidas descriptivas de las tres métricas RAGAS y del tiempo de respuesta para las tres configuraciones del grupo y para el sistema de referencia. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación.

Tabla 5.4. Estadísticas descriptivas del experimento de *retriever* híbrido.

Métrica	Estadístico	CS-1000	H-50/50	H-30/70	H-70/30
<i>Faithfulness</i>	Media	0,349	0,472	0,456	0,435
<i>Faithfulness</i>	Mediana	0,333	0,500	0,444	0,437
<i>Faithfulness</i>	Desv. estándar	0,248	0,304	0,312	0,270
<i>Response Relevancy</i>	Media	0,607	0,599	0,608	0,615
<i>Response Relevancy</i>	Mediana	0,621	0,604	0,618	0,599
<i>Response Relevancy</i>	Desv. estándar	0,129	0,127	0,133	0,097
<i>Context Precision</i>	Media	0,846	0,833	0,809	0,861
<i>Context Precision</i>	Mediana	0,991	0,939	0,934	0,966
<i>Context Precision</i>	Desv. estándar	0,244	0,233	0,254	0,225
Tiempo RAG (s)	Media	16,700	6,390	6,560	6,340
Tiempo RAG (s)	Mediana	6,130	6,070	6,490	5,960
Tiempo RAG (s)	Desv. estándar	80,620	2,110	1,590	1,770

Las tres configuraciones híbridas alcanzan medianas de *Faithfulness* entre 0,437 y 0,500, claramente por encima de la del sistema de referencia (0,333), con H-50/50 como mejor valor del grupo. En *Response Relevancy* las cuatro configuraciones obtienen medianas muy similares, en un rango estrecho alrededor de 0,600. En *Context Precision* las tres configuraciones híbridas obtienen medianas entre 0,934 y 0,966, ligeramente por debajo de la del sistema de referencia (0,991), aunque todas se mantienen en valores altos dentro del rango de la métrica. La mediana del tiempo de respuesta es prácticamente la misma en todo el grupo, alrededor de 6 segundos por consulta, lo que indica que la búsqueda léxica BM25 añadida a la búsqueda densa no incrementa el coste del *pipeline* de forma notable.

El test global de Friedman aplicado al grupo no detecta diferencias significativas en ninguna de las tres métricas, con p-values altos en todos los casos (*Faithfulness* $p = 0,456$, *Response Relevancy* $p = 0,516$, *Context Precision* $p = 0,782$). Las comparaciones por pares con el sistema de referencia tampoco resultan significativas tras la corrección de Holm. El resultado más cercano al umbral es la mejora de H-50/50 en *Faithfulness* con un p-value sin corregir de 0,046 y un efecto mediano ($r = 0,31$), que pasa a $p = 0,139$ tras Holm y no entra ni en la categoría de significancia ni en la de tendencia definida en la sección 4.5. Las otras dos configuraciones también

presentan efectos pequeños y positivos en *Faithfulness* ($r = 0,28$ y $r = 0,25$) que tampoco resisten la corrección.

Las comparaciones por pares dentro del grupo confirman la ausencia de diferencias entre los tres pesos evaluados. Ningún par de configuraciones difiere de forma significativa en ninguna métrica tras la corrección de Holm, y todos los efectos quedan en el rango de pequeño o insignificante. Los resultados sugieren que, en este experimento, la proporción concreta entre búsqueda semántica y BM25 no influye de forma medible en la calidad final del sistema.

El experimento no produce mejoras significativas frente al sistema de referencia en ninguna de las tres métricas, aunque tampoco incrementa el coste del *pipeline*. Los valores de *Context Precision* en el grupo se mantienen en el mismo rango alto que el sistema de referencia, lo que indica que la incorporación de BM25 no empeora la calidad del *retriever* pero tampoco la mejora. Una posible explicación es que las consultas evaluadas utilizan un vocabulario relativamente cercano al del corpus, lo que reduce los casos en los que la coincidencia léxica exacta de BM25 puede aportar fragmentos que la búsqueda densa no haya recuperado ya. En *Faithfulness* se observa una tendencia descriptiva consistente, con las tres configuraciones híbridas por encima del sistema de referencia en sus medianas, aunque ninguna de las diferencias alcanza el umbral de significancia tras la corrección. Estos resultados se discuten junto con el resto de experimentos en la sección 5.7.

5.4. *Multi-Query*

El tercer experimento evalúa el enfoque *Multi-Query*, en el que la consulta original se amplía con varias reformulaciones generadas por el propio modelo antes de la búsqueda, siguiendo el esquema descrito en la sección 2.3. La hipótesis que motiva el experimento es que distintas formulaciones de la misma pregunta puedan recuperar fragmentos diferentes del corpus, ampliando la cobertura del *retriever* y reduciendo la sensibilidad a la redacción concreta del usuario. La implementación utiliza `MultiQueryRetriever` de `LangChain`, que genera N subconsultas mediante un *prompt* aplicado a la llama3.1:8b (el texto del *prompt* se recoge en el Anexo B.4), realiza una búsqueda densa independiente para cada una con k fragmentos, y combina los resultados deduplicando los fragmentos repetidos antes de inyectarlos en el *prompt* del generador. La figura 5.2 ilustra el mecanismo.

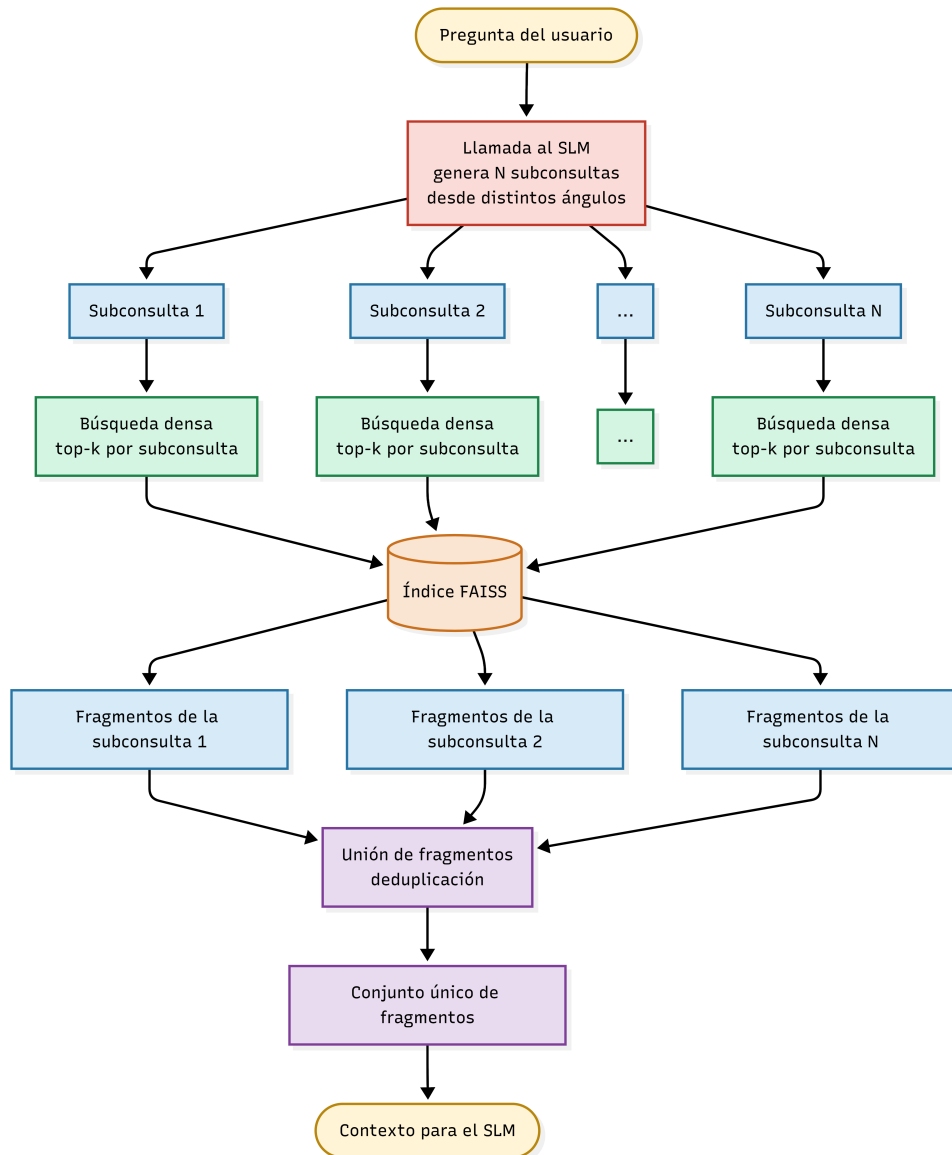


Figura 5.2. Mecanismo de *Multi-Query retrieval*.

Las dos configuraciones se diseñan de modo que el resultado de $N*k$ sea igual a 12 fragmentos antes de la deduplicación, lo que después de eliminar los duplicados produce alrededor de 10 fragmentos finales en la inyección al *prompt*. La tasa de deduplicación aproximada se estimó empíricamente en pruebas previas, y el valor de 12 fragmentos en bruto se eligió para que el contexto final mantenga el tamaño de unos 10.000 caracteres que sirve de invariante en todos los experimentos del capítulo. Las dos configuraciones evaluadas son:

- MQ-B4x3; con 4 subconsultas y $k=3$ fragmentos por cada una
- MQ-B3x4; con 3 subconsultas y $k=4$ fragmentos por cada una.

La tabla 5.5 recoge las medidas descriptivas de las tres métricas RAGAS y del tiempo de respuesta para las dos configuraciones del grupo y para el sistema de referencia. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación.

Tabla 5.5. Estadísticas descriptivas del experimento *Multi-Query*.

Métrica	Estadístico	CS-1000	MQ-B4x3	MQ-B3x4
<i>Faithfulness</i>	Media	0,349	0,390	0,359
<i>Faithfulness</i>	Mediana	0,333	0,375	0,308
<i>Faithfulness</i>	Desv. estándar	0,248	0,249	0,302
<i>Response Relevancy</i>	Media	0,607	0,606	0,614
<i>Response Relevancy</i>	Mediana	0,621	0,625	0,609
<i>Response Relevancy</i>	Desv. estándar	0,129	0,180	0,164
<i>Context Precision</i>	Media	0,846	0,855	0,817
<i>Context Precision</i>	Mediana	0,991	1,000	0,984
<i>Context Precision</i>	Desv. estándar	0,244	0,253	0,309
Tiempo RAG (s)	Media	16,700	7,770	7,750
Tiempo RAG (s)	Mediana	6,130	7,630	7,890
Tiempo RAG (s)	Desv. estándar	80,620	1,490	1,540

Los datos de la tabla muestran valores muy cercanos a los del sistema de referencia en las tres métricas RAGAS. MQ-B4x3 obtiene medianas ligeramente por encima del sistema de referencia en *Faithfulness* (0,375 frente a 0,333) y en *Context Precision* (1,000 frente a 0,991), y prácticamente iguales en *Response Relevancy*. MQ-B3x4 queda por debajo del sistema de referencia en *Faithfulness* (0,308 frente a 0,333) y muy cerca en las otras dos métricas. La mediana del tiempo de respuesta sube alrededor de un segundo y medio respecto al sistema de referencia (7,630 y 7,890 frente a 6,130), por la llamada adicional al modelo de lenguaje que genera las subconsultas.

Como el grupo contiene únicamente dos configuraciones, no se aplica el test de Friedman descrito en la sección 4.5, y se realizan directamente las comparaciones por pares con Wilcoxon frente al sistema de referencia. Ninguna de las seis comparaciones (dos configuraciones por tres métricas) alcanza significancia, ni siquiera antes de la corrección de Holm. Los p-values sin corregir se sitúan entre 0,371 y 0,927, y los tamaños del efecto son pequeños o insignificantes en todos los casos ($|r| \leq 0,14$). Ninguna de las comparaciones entra en la categoría de tendencia definida en la sección 4.5.

Entre las dos configuraciones del grupo, MQ-B4x3 (cuatro subconsultas con $k=3$) muestra valores descriptivamente algo mejores que MQ-B3x4 (tres subconsultas con

$k=4$) en las tres métricas, lo que sugiere que aumentar el número de reformulaciones a costa de reducir los fragmentos recuperados por cada una resulta más útil que la combinación opuesta. Sin embargo, la diferencia entre ambas es demasiado pequeña para extraer conclusiones, en parte porque los valores explorados de N y k (3 y 4) están muy próximos entre sí y limitan la amplitud del contraste. Una exploración con configuraciones de $N*k$ más separadas (por ejemplo, $2*6$ frente a $6*2$) permitiría verificar si esta tendencia se confirma, aunque requeriría ajustar el invariante de contexto de 10.000 caracteres definido en la sección 4.3, por lo que se propone como posible línea de trabajo futura.

El experimento no produce mejoras significativas frente al sistema de referencia en ninguna de las tres métricas, y aumenta la mediana del tiempo de respuesta alrededor de un 25 % por la generación adicional de subconsultas. Una posible explicación es que las reformulaciones generadas por la llama3.1:8b sobre un corpus temáticamente homogéneo no introducen suficiente diversidad léxica para que la búsqueda densa recupere fragmentos distintos a los que ya obtendría con la consulta original. Las métricas se mantienen en el mismo rango, lo que sugiere que, tras la deduplicación, el conjunto final de fragmentos coincide en gran parte con el del sistema de referencia. Estos resultados se discuten junto con el resto de experimentos en la sección 5.7.

5.5. *Rewrite-Retrieve-Read*

El siguiente experimento evalúa el efecto de añadir un paso de reformulación de la consulta entre la primera recuperación y la generación final, siguiendo el esquema inspirado en *Rewrite-Retrieve-Read* descrito en la sección 2.3. En lugar de buscar directamente a partir de la pregunta del usuario, el sistema realiza primero una recuperación amplia, utiliza los fragmentos obtenidos para reformular la consulta y, con la nueva versión de la pregunta, hace una segunda recuperación que proporciona el contexto al generador. La hipótesis que motiva el experimento es que la consulta reformulada se alinee mejor con el vocabulario del corpus y mejore la precisión del contexto recuperado y la calidad de la respuesta. La implementación de este trabajo no reproduce el RRR original de Ma et al. (2023), que entrena un modelo específico para la reformulación, sino más bien una variante simplificada en la que la propia llama3.1:8b actúa como reformuladora a partir de una instrucción en lenguaje natural. La figura 5.3 ilustra el mecanismo.

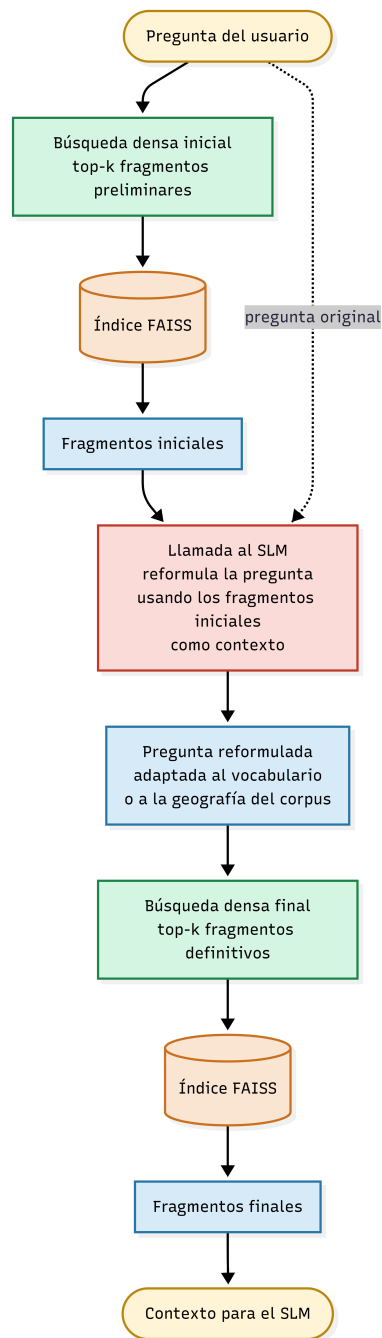


Figura 5.3. Mecanismo de *Rewrite-Retrieve-Read*.

En el esquema utilizado existen dos elementos que se pueden modificar: el *prompt* que da las instrucciones de reformulación y la cantidad de contexto recuperado en el primer paso que se le entrega a ese *prompt*. Pruebas preliminares con distintos tamaños de contexto inicial no mostraron diferencias notables, por lo que se decidió explorar también la variación del *prompt* de reformulación. Las tres configuraciones evaluadas son:

- RRR-AD-300: un *prompt* de adaptación al vocabulario del corpus y los primeros 300 caracteres de cada uno de los 6 fragmentos iniciales como contexto del paso de reformulación
- RRR-AD: un *prompt* de adaptación al vocabulario del corpus pero usando los 10 fragmentos iniciales completos
- RRR-GE: un *prompt* orientado a precisar la geografía y los parámetros concretos de la consulta (provincias, comarcas, municipios, espacios naturales), también con los 10 fragmentos iniciales completos.

Los textos completos de los *prompts* de reformulación utilizados en estas tres configuraciones se recogen en el Anexo B (apartados B.5 y B.6).

En las tres configuraciones la recuperación final se realiza sobre el mismo índice FAISS del sistema de referencia con $k=10$, de modo que el contexto que llega al generador conserva el volumen de 10.000 caracteres.

La tabla 5.6 recoge las medidas descriptivas de las tres métricas RAGAS y del tiempo de respuesta para las tres configuraciones del grupo y para el sistema de referencia. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación.

Tabla 5.6. Estadísticas descriptivas del experimento *Rewrite-Retrieve-Read*.

Métrica	Estadístico	CS-1000	RRR-AD-300	RRR-AD	RRR-GE
Faithfulness	Media	0,349	0,387	0,384	0,435
Faithfulness	Mediana	0,333	0,358	0,400	0,408
Faithfulness	Desv. estándar	0,248	0,254	0,173	0,271
Response Relevancy	Media	0,607	0,613	0,614	0,595
Response Relevancy	Mediana	0,621	0,626	0,641	0,606
Response Relevancy	Desv. estándar	0,129	0,120	0,105	0,153
Context Precision	Media	0,846	0,878	0,890	0,842
Context Precision	Mediana	0,991	1,000	0,963	0,973
Context Precision	Desv. estándar	0,244	0,235	0,180	0,268
Tiempo RAG (s)	Media	16,700	6,910	7,580	8,350
Tiempo RAG (s)	Mediana	6,130	6,900	7,610	8,360
Tiempo RAG (s)	Desv. estándar	80,620	1,230	0,990	1,740

Como se observa en la tabla, las tres configuraciones de RRR producen medianas de *Faithfulness* por encima del sistema de referencia (entre 0,358 y 0,408 frente a 0,333), con RRR-GE en la posición más alta. En *Response Relevancy* las cuatro configuraciones se mantienen en un rango estrecho alrededor de 0,621, con RRR-AD ligeramente por encima del resto y RRR-GE ligeramente por debajo. *Context*

Precision no muestra cambios relevantes, ya que las cuatro medianas se sitúan por encima de 0,96. La mediana del tiempo de respuesta sube entre 0,8 y 2,2 segundos respecto al sistema de referencia, lo que se corresponde con el coste de añadir una llamada adicional al modelo de lenguaje en el paso de reformulación.

El test global de Friedman aplicado al grupo encuentra diferencias significativas en *Response Relevancy* ($\chi^2 = 10,39$, $p = 0,006$), pero no en *Faithfulness* ($p = 0,886$) ni en *Context Precision* ($p = 0,754$). Sin embargo, las comparaciones por pares con el sistema de referencia no llegan a ser significativas en ninguna de las tres configuraciones y en ninguna de las tres métricas tras la corrección de Holm. La mejora más cercana al umbral es la de RRR-GE en *Faithfulness* ($p = 0,059$ sin corregir, $p = 0,176$ tras Holm; $r = 0,29$, efecto pequeño), que tampoco entra en la categoría de tendencia definida en la sección 4.5 ya que el p-value corregido se aleja del intervalo entre 0,05 y 0,10. Las mejoras observadas en la tabla de *Faithfulness* y en *Context Precision* dejan de ser estadísticamente significativas tras aplicar la corrección por comparaciones múltiples.

Las comparaciones por pares dentro del grupo señalan a *Response Relevancy* como la métrica que explica el resultado de Friedman. Las comparaciones por pares donde se evalúa RRR-GE presentan valores muy cercanos al umbral de aceptación antes de corregir, ya que RRR-AD-300 frente a RRR-GE alcanza $p = 0,048$ ($r = 0,30$) y RRR-GE frente a RRR-AD logra $p = 0,051$ ($r = -0,29$). Tras aplicar Holm dentro del grupo, ambas pasan a $p = 0,144$ y dejan de ser significativas. Se observa que RRR-GE, con su *prompt* geográfico, muestra un comportamiento distinto frente a las variantes de adaptación al obtener la mediana de *Faithfulness* más alta del grupo junto con la de *Response Relevancy* más baja. Una posible explicación es que, al reformular la consulta para que sea más específica geográficamente, el generador produce respuestas más fieles al contexto pero menos alineadas con la pregunta original. El test de Friedman detecta este efecto, pero la corrección por comparaciones múltiples no permite atribuirlo formalmente a una configuración concreta.

El experimento no produce mejoras significativas frente al sistema de referencia en ninguna de las tres métricas, pero incrementa el coste del *pipeline* entre un 13 % y un 36 % por la llamada adicional al modelo en el paso de reformulación. Varias circunstancias del entorno pueden explicar la ausencia de mejora medible. Por un lado, la reformulación se realiza con el mismo modelo llama3.1:8b que genera la respuesta final. Un modelo de este tamaño puede tener dificultad para reescribir la consulta con cambios léxicos suficientes para que la búsqueda densa recupere fragmentos distintos, sobre todo si la consulta original ya usa un vocabulario parecido al del

corpus. Por otro lado, al ser un corpus temáticamente uniforme (rutas y espacios naturales de Andalucía), se reduce la posibilidad de que la reformulación añada información útil que mejore la recuperación original. Estos resultados se discuten junto con el resto de los experimentos en la sección 5.7.

5.6. Metadatos

El último experimento del capítulo evalúa el efecto de enriquecer cada fragmento del corpus con un conjunto de metadatos estructurados antes de su indexación. La hipótesis que motiva el experimento es que la información estructurada al inicio de cada fragmento (provincia, municipios, dificultad, tipo de trayecto y necesidad de autorización especial) facilita al *retriever* la recuperación de los fragmentos más útiles cuando la consulta del usuario menciona alguno de esos atributos.

En esta implementación los metadatos se procesan como texto. Los valores de los metadatos se agrupan en una cabecera con formato fijo y se anteponen al contenido de cada fragmento antes del cálculo del *embedding*, de modo que el vector resultante recoge la cabecera y el texto. Esta vía es una alternativa manual a la filtración estructurada por metadatos, que requiere una base de datos vectorial con soporte nativo de filtros (Qdrant, Weaviate o Chroma, etc.). FAISS, el almacén utilizado en el resto del trabajo, no dispone de este tipo de filtrado de forma directa, y el cambio de base de datos habría modificado un componente del sistema que se mantiene fijo en todos los experimentos.

La extracción de los cinco campos se realizó mediante expresiones regulares aplicadas a la primera página de cada PDF, donde la mayoría de los documentos del corpus tienen una ficha técnica con un formato relativamente estable. El proceso se desarrolló en varias iteraciones, partiendo de un conjunto inicial de patrones y ampliándolo con variantes en español e inglés hasta alcanzar coberturas por campo entre el 78 % y el 97 % sobre 641 PDFs con texto extraíble. Al mismo tiempo, se exploró una alternativa basada en llama3.1:8b con *prompt* estructurado, que con la opción `format="json"` de Ollama alcanza una cobertura cercana al 99 % en todos los campos. Sin embargo, esta vía fue descartada, porque una revisión parcial de las respuestas del modelo detectó valores alucinados, mientras que las expresiones regulares garantizan que los valores extraídos están en el texto fuente.

La configuración evaluada, denominada META-B, mantiene los parámetros del sistema de referencia (*chunk* de 1.000 caracteres, solapamiento de 200, $k=10$) y se diferencia únicamente en que se construye un nuevo índice FAISS en el que los fragmentos procedentes de PDFs tienen al inicio la cabecera de metadatos. Un análisis

sobre el índice resultante muestra que un 97,6 % de los fragmentos llevan cabecera, lo que confirma que la inyección se aplicó de forma correcta. El resto del *pipeline* (modelo, *embeddings*, *prompt* del sistema y protocolo de evaluación) es idéntico al del sistema de referencia.

La tabla 5.7 recoge las medidas descriptivas de las tres métricas RAGAS y del tiempo de respuesta para META-B y para el sistema de referencia. Los valores corresponden a las medianas por pregunta agregadas sobre las 60 preguntas del conjunto de evaluación.

Tabla 5.7. Estadísticas descriptivas del experimento de inyección de metadatos.

Métrica	Estadístico	CS-1000	META-B
<i>Faithfulness</i>	Media	0,349	0,372
<i>Faithfulness</i>	Mediana	0,333	0,333
<i>Faithfulness</i>	Desv. estándar	0,248	0,299
<i>Response Relevancy</i>	Media	0,607	0,589
<i>Response Relevancy</i>	Mediana	0,621	0,618
<i>Response Relevancy</i>	Desv. estándar	0,129	0,164
<i>Context Precision</i>	Media	0,846	0,766
<i>Context Precision</i>	Mediana	0,991	0,906
<i>Context Precision</i>	Desv. estándar	0,244	0,303
Tiempo RAG (s)	Media	16,700	37,540
Tiempo RAG (s)	Mediana	6,130	5,680
Tiempo RAG (s)	Desv. estándar	80,620	137,480

La métrica con un cambio más visible es *Context Precision*, en la cual la mediana cae de 0,991 en el sistema de referencia a 0,906 en META-B, mientras que la desviación estándar aumenta de 0,244 a 0,303. *Faithfulness* mantiene la misma mediana (0,333) y *Response Relevancy* queda prácticamente igual (0,618 frente a 0,621). La mediana del tiempo de respuesta no aumenta respecto al sistema de referencia, lo que es esperable dado que la cabecera se incorpora en el momento de la indexación y no añade pasos al *pipeline* en tiempo de consulta.

Como el grupo contiene únicamente una configuración frente al sistema de referencia, no se aplica el test de Friedman. Las comparaciones por pares con Wilcoxon muestran que la caída de *Context Precision* está cerca del umbral de la tendencia definida en la sección 4.5, con $p = 0,052$ tras la corrección de Holm y un efecto mediano ($r = -0,33$). Aunque este valor no alcanza la significancia estadística estricta, sugiere un empeoramiento sistemático en esta métrica. *Faithfulness* y *Response*

Relevancy no presentan diferencias significativas en comparación con el sistema de referencia, y los tamaños del efecto son insignificantes en ambos casos.

El experimento no produce las mejoras esperadas y muestra una tendencia ligera al empeoramiento en *Context Precision*, la métrica que evalúa la calidad del orden con que el *retriever* devuelve los fragmentos. Una posible explicación combina dos factores. Por un lado, dado que la cabecera se construye con los metadatos de la primera página de cada PDF y se añade de forma idéntica en todos los fragmentos del documento, los vectores de fragmentos del mismo PDF se acercan entre sí en el espacio vectorial. Esta homogeneización reduce la capacidad del *retriever* para distinguir el fragmento más relevante dentro de un mismo documento. Por otro lado, solo una parte de las preguntas evaluadas hace referencia directa a alguno de los cinco campos inyectados, por lo que en el resto de las consultas la cabecera funciona como información añadida sin correspondencia con la pregunta y puede distorsionar la similitud sin aportar señal útil. Los resultados sugieren que la incorporación de metadatos sería más adecuada a través de una base de datos vectorial con filtrado nativo por campos. Estos resultados se discuten junto con el resto de los experimentos en la sección 5.7.

5.7. Discusión global

Esta sección recoge los resultados de todos los experimentos y los interpreta en relación con los objetivos planteados en el capítulo 1. Las once configuraciones experimentales evaluadas, junto con el sistema de referencia, se resumen en la tabla 5.8, que permite comparar a la vez las medianas de las tres métricas RAGAS y la significancia de cada comparación con el sistema de referencia tras la corrección de Holm.

Tabla 5.8. Resumen de las configuraciones evaluadas.

Grupo	Config.	<i>Faith.</i>	sig.	<i>Resp. Rel.</i>	sig.	<i>Ctx. Prec.</i>	sig.
Referencia	CS-1000	0,333	–	0,621	–	0,991	–
<i>Chunk size</i>	CS-500	0,431	~	0,630	ns	1,000	*
<i>Chunk size</i>	CS-2000	0,387	ns	0,624	ns	0,845	*
<i>Hybrid</i>	H-50/50	0,500	ns	0,604	ns	0,939	ns
<i>Hybrid</i>	H-30/70	0,444	ns	0,618	ns	0,934	ns
<i>Hybrid</i>	H-70/30	0,437	ns	0,599	ns	0,966	ns
<i>Multi-Query</i>	MQ-B4x3	0,375	ns	0,625	ns	1,000	ns
<i>Multi-Query</i>	MQ-B3x4	0,308	ns	0,609	ns	0,984	ns
RRR	RRR-AD-300	0,358	ns	0,626	ns	1,000	ns
RRR	RRR-AD	0,400	ns	0,641	ns	0,963	ns
RRR	RRR-GE	0,408	ns	0,606	ns	0,973	ns
Metadatos	META-B	0,333	ns	0,618	ns	0,906	~

Nota: La significancia (sig.) corresponde a la comparación con el sistema de referencia tras la corrección de Holm. El símbolo * indica un resultado significativo ($p < 0,05$); el símbolo ~ indica una tendencia ($0,05 \leq p \leq 0,10$) con un tamaño del efecto mediano o grande ($|r| \geq 0,3$). El valor ns recoge el resto de los casos.

Como se observa en la tabla, el experimento sobre el tamaño de *chunk* es el único que produce mejoras estadísticamente significativas respecto al sistema de referencia. Reducir el tamaño de *chunk* de 1.000 a 500 caracteres mejora *Context Precision* con un efecto grande ($r = 0,51$), mientras que aumentarlo a 2.000 lo empeora con un efecto mediano ($r = -0,44$). Los tres pares intragrupo en esta métrica resultan significativos y apuntan en la misma dirección, lo que permite ordenar las tres configuraciones de mayor a menor calidad de recuperación: CS-500, CS-1000 y CS-2000. En *Faithfulness* se observa una tendencia similar ($r = 0,33$ con $p = 0,056$ tras la corrección), aunque no alcance el umbral formal.

Los cuatro experimentos restantes (*retriever* híbrido, *Multi-Query*, *Rewrite-Retrieve-Read* e inyección de metadatos) no producen mejoras significativas frente al sistema de referencia en ninguna métrica tras la corrección de Holm. Los tamaños del efecto se quedan en el rango pequeño o insignificante en la mayor parte de los casos, y los únicos resultados que se acercan al umbral son los efectos pequeños positivos del grupo *Hybrid* en *Faithfulness* y el efecto mediano negativo de META-B en *Context Precision*. En particular, las tres configuraciones del grupo *Hybrid* presentan las

medianas más altas de *Faithfulness* de todo el conjunto de experimentos (entre 0,437 y 0,500 frente a 0,333 del sistema de referencia), pero el comportamiento ruidoso de esta métrica dentro de cada configuración impide que la mejora descriptiva alcance el umbral de significancia. META-B muestra una tendencia al empeoramiento en *Context Precision* ($r = -0,33$ con $p = 0,052$ tras la corrección).

Al examinar estos cuatro experimentos en conjunto, se distinguen dos comportamientos según el tipo de modificación introducida en el *pipeline*. Las técnicas que añaden llamadas al modelo de lenguaje, como RRR y *Multi-Query*, incrementan la mediana del tiempo de respuesta entre un 13 % y un 36 % en RRR y alrededor de un 25 % en *Multi-Query*, sin que este incremento conlleve mejoras estadísticamente significativas en ninguna métrica. Las técnicas que actúan sobre el índice o el *retriever* sin añadir nuevas llamadas al modelo, como *Hybrid* y la inyección de metadatos, no aumentan el tiempo de respuesta pero tampoco aportan mejoras significativas. La consecuencia práctica para este trabajo es que el principio de mantener el *pipeline* lo más simple posible se confirma con los resultados de este trabajo.

La ausencia de mejoras significativas en las técnicas evaluadas puede deberse a una combinación de dos factores cuya contribución relativa no es posible separar dentro del diseño del estudio. Por un lado, el SLM de 8.000 millones de parámetros utilizado como modelo único para generación, cálculo de *embeddings* y evaluación tiene una capacidad limitada para producir reformulaciones de consulta o subconsultas con suficiente diversidad léxica como para alterar de forma apreciable los resultados de la búsqueda densa. Por otro lado, el corpus utilizado es temáticamente homogéneo, centrado en rutas y espacios naturales de Andalucía, con un vocabulario común en la mayoría de los documentos. Esta homogeneidad reduce el margen para que cualquier técnica basada en aportar variabilidad léxica (RRR y *Multi-Query*) o señal estructural adicional (*Hybrid* y metadatos) recupere fragmentos distintos a los que el sistema de referencia ya obtiene con la consulta original. Separar la contribución de cada factor exigiría experimentos adicionales con otras combinaciones de modelo y corpus, planteados entre las líneas de trabajo futuro del capítulo 6.

La comparación conjunta de las doce configuraciones también permite extraer algunas observaciones sobre el propio comportamiento de las métricas RAGAS en este sistema. *Context Precision* resulta ser la métrica más sensible a las modificaciones del *retriever*, ya que es la única en la que se obtienen diferencias significativas respecto al sistema de referencia, tanto de aumento (CS-500) como de caída (CS-2000), además de una tendencia al empeoramiento en META-B. *Faithfulness* se mantiene en niveles bajos en el sistema de referencia (mediana de 0,333) y muestra un claro potencial para crecer, pero ninguna técnica logra mejorarla de forma significativa

tras la corrección, aunque sí aparecen tendencias y efectos pequeños positivos en varios experimentos. *Response Relevancy* se queda prácticamente plana en las once configuraciones experimentales y en el sistema de referencia, con medianas entre 0,599 y 0,641, lo que sugiere que su valor depende menos del *retriever* y más de la capacidad del propio modelo de generar respuestas alineadas con la pregunta.

En relación con los objetivos planteados en 1.3, el conjunto de experimentos cumple con la tarea de evaluar el rendimiento del sistema ante distintas configuraciones y de extraer una recomendación concreta para el caso de uso del proyecto NATURCOR-IA. La configuración con *chunk* de 500 caracteres y $k=20$ es la opción más adecuada entre las evaluadas para el corpus actual, ya que mejora la calidad del orden con que el *retriever* devuelve los fragmentos útiles sin empeorar el resto de las métricas ni el tiempo de respuesta. El uso de las técnicas más elaboradas de la familia RAG Avanzado evaluadas en este trabajo (*Hybrid*, *Multi-Query* y *RRR*) no conlleva mejoras medibles sobre este sistema concreto, lo que en sí mismo constituye un resultado informativo, ya que aporta evidencia, dentro de los límites del estudio, de que la simplicidad del sistema base no es un obstáculo para alcanzar la mejor calidad observada en las pruebas. Las limitaciones que condicionan el alcance de estas conclusiones y las líneas de continuación se desarrollan en el capítulo 6.

Conclusiones y trabajo futuro

6.1. Conclusiones

Este Trabajo Fin de Máster ha desarrollado y evaluado un sistema conversacional basado en la arquitectura RAG con un modelo de lenguaje pequeño de ejecución local, orientado a facilitar el acceso a la información sobre rutas y espacios naturales de la Junta de Andalucía dentro del proyecto NATURCOR-IA. El trabajo cubre todas las etapas del sistema, desde la construcción del corpus documental hasta la evaluación experimental de distintas configuraciones del *pipeline*. Las conclusiones recogen los resultados sobre el sistema construido y las aportaciones metodológicas del estudio.

El trabajo ha permitido desarrollar un sistema RAG funcional de ejecución local. La ingesta cubre dos tipos de fuentes, los PDFs del corpus de senderos de Andalucía y un conjunto de páginas web seleccionadas. El corpus se segmenta en fragmentos, se transforma en *embeddings* con llama3.1:8b a través de Ollama y se indexa en FAISS. La consulta se resuelve mediante búsqueda densa sobre el índice y generación de respuesta con el mismo modelo, con memoria conversacional y un *prompt* de sistema orientado a reducir las alucinaciones. La ejecución local responde a uno de los requisitos del proyecto NATURCOR-IA, ya que evita el coste recurrente asociado al uso de servicios de inferencia externos e independiza el sistema de los cambios en las APIs y políticas de los proveedores comerciales.

Junto al sistema, el trabajo ha definido un protocolo experimental reproducible y un marco estadístico para comparar configuraciones del *pipeline*. Las doce configuraciones evaluadas se dividen en cinco grupos (tamaño de *chunk*, *retriever* híbrido, *Multi-Query*, *Rewrite-Retrieve-Read* e inyección de metadatos) y cada una se prueba con 60 preguntas repetidas en 10 iteraciones, lo que da un total de 7.200 evaluaciones individuales. La evaluación se realiza con RAGAS sin respuestas de referencia, agregando los datos a nivel de pregunta calculando la mediana y aplicando un protocolo no paramétrico (test de Friedman, Wilcoxon por pares con corrección de Holm y tamaños del efecto según los umbrales de Cohen). El marco permite separar las diferencias reales del ruido propio del modelo.

La evaluación identifica el tamaño de *chunk* como el único parámetro con un efecto estadísticamente significativo sobre la calidad del sistema entre los evaluados. La configuración con *chunk* de 500 caracteres y $k=20$ mejora *Context Precision* frente al sistema de referencia con un efecto grande, mientras que la configuración con *chunk* de 2.000 caracteres y $k=5$ lo empeora con un efecto mediano, en ambos casos con significancia tras la corrección por comparaciones múltiples. Las técnicas más elaboradas evaluadas (*retriever* híbrido, *Multi-Query*, *Rewrite-Retrieve-Read* e inyección de metadatos) no producen mejoras significativas respecto al sistema de referencia, y las que añaden llamadas adicionales al modelo de lenguaje (RRR y *Multi-Query*) incrementan además el coste temporal del *pipeline* sin un beneficio correspondiente en la calidad medida.

Entre las configuraciones evaluadas en este trabajo, la opción con *chunk* de 500 caracteres y $k=20$ es la que mejor se comporta de forma global y se propone como punto de partida para la integración del sistema RAG en el proyecto NATURCOR-IA. La recomendación se limita a las condiciones del estudio: corpus temático sobre rutas y espacios naturales de Andalucía, modelo llama3.1:8b, evaluación sobre 60 preguntas representativas del caso de uso y métricas RAGAS sin respuestas de referencia. Una validación con usuarios reales del sistema, una vez integrado en la plataforma web del proyecto, permitiría confirmar o matizar esta recomendación mediante criterios de calidad complementarios a los utilizados en este estudio.

Además del resultado concreto para NATURCOR-IA, el trabajo propone una metodología de evaluación de sistemas RAG que distingue entre diferencias reales y aleatorias en un entorno con variabilidad no determinista considerable. La combinación de iteraciones múltiples por pregunta, agregación mediante la mediana, tests no paramétricos con corrección por comparaciones múltiples y reporte del tamaño del efecto es aplicable a otros sistemas RAG sobre SLMs locales y puede reutilizarse en evaluaciones posteriores de este tipo de arquitecturas.

6.2. Limitaciones

El sistema utiliza un único modelo (llama3.1:8b) para tres tareas distintas: la generación de respuestas, el cálculo de *embeddings* del corpus y de las consultas, y la evaluación automática mediante RAGAS. Esta elección se justificó en su momento por la coherencia entre componentes y la simplicidad del despliegue, pero deja fuera del estudio dos aspectos relevantes. Los modelos *decoder-only* sin ajuste fino específico no están optimizados para la tarea de *embeddings*, y alternativas *encoder* como BGE o E5 obtienen en general mejores resultados en recuperación semántica, como se señala en la sección 3.3.3. Además, los 8 GB de VRAM del servidor utilizado limitan el tamaño del modelo que puede ejecutarse en local, lo que impide evaluar variantes más grandes de la misma familia (13B o 70B) o modelos de otras familias del mismo orden que requieran más memoria.

La principal limitación del protocolo de evaluación es que el modelo evaluado y el modelo juez son el mismo. Este efecto, conocido como *self-preference bias* (Panickssery et al., 2024) y mencionado en la sección 4.1, hace que las puntuaciones de RAGAS tiendan a favorecer las respuestas generadas con el mismo modelo que las evalúa, lo que introduce un sesgo en los valores absolutos de las métricas. El diseño del estudio reduce parcialmente este efecto al centrarse en comparaciones entre configuraciones que comparten el mismo juez, pero el sesgo permanece en los valores observados. Además, las métricas RAGAS empleadas son *reference-free*, es decir, evalúan cada respuesta a partir de la pregunta y del contexto recuperado, sin compararla con una respuesta de referencia anotada manualmente. Este enfoque resulta práctico cuando no existe *ground truth* disponible, pero la calidad de la evaluación depende de la capacidad del propio modelo juez para juzgar el contenido. El conjunto de evaluación consta de 60 preguntas. El diseño con 10 iteraciones por pregunta y la agregación por mediana aporta estabilidad a las estimaciones, pero una muestra mayor podría permitir detectar como significativas algunas de las tendencias que en este estudio quedan por debajo del umbral tras la corrección por comparaciones múltiples.

El experimento de inyección de metadatos presenta dos limitaciones propias de la implementación. La incorporación de los metadatos al sistema se realizó como una cabecera textual añadida a cada fragmento antes del cálculo del *embedding* y no como una capa de filtrado nativo por campos. Esta alternativa habría requerido sustituir FAISS por un almacén vectorial con capacidad de filtrado por atributos, opción incompatible con el principio de mantener fijos el resto de componentes del *pipeline* a lo largo de los experimentos del capítulo 5. La extracción de los propios metadatos también está condicionada por las características del corpus. Los PDFs presentan

una estructura similar entre sí pero no estrictamente uniforme, con variaciones en el formato de la ficha técnica, el orden de los campos y la presencia o ausencia de cada uno de ellos. Esta variabilidad limita la cobertura alcanzable por una extracción basada en expresiones regulares, que en este trabajo se sitúa entre el 78 % y el 97 % según el campo.

Cada una de estas limitaciones marca una dirección concreta en la que el sistema o el protocolo de evaluación pueden extenderse y, en ese sentido, sirven de punto de partida para las líneas de trabajo futuro que se desarrollan en la sección siguiente.

6.3. Líneas de trabajo futuro

La mejora más directa sobre el sistema actual es la sustitución del modelo de *embeddings* por un *encoder* especializado. Como se comenta en la sección 3.3.3, alternativas como BGE o E5 están diseñadas para la tarea de recuperación y obtienen mejores resultados que un *decoder* generalista usado fuera de su propósito principal. El cambio afecta a la fase del *pipeline* donde se concentra el resultado significativo del capítulo 5 y no requiere modificar el resto de los componentes. Junto a este cambio, también sería conveniente reemplazar FAISS por un almacén vectorial con soporte nativo de filtros por campos, como Qdrant, Weaviate o Chroma, para reintentar la incorporación de metadatos por la vía del filtrado estructurado en lugar de la inyección textual evaluada en META-B.

Asimismo, repetir los experimentos con un modelo generador más grande que llama3.1:8b permitiría aislar la contribución del tamaño del modelo en cada técnica evaluada. La interpretación del capítulo 5 atribuye la ausencia de mejoras significativas en las técnicas RAG Avanzado a una combinación entre la capacidad limitada de un SLM de 8.000 millones de parámetros y la homogeneidad léxica del corpus, sin que el diseño del estudio permita separar ambos factores. Además, otros experimentos que quedaron fuera del alcance de este trabajo por el principio de modificar un único parámetro por experimento serían las configuraciones de Multi-Query con mayor amplitud entre N y k , y la evaluación de combinaciones entre parámetros, como el *chunk* de 500 caracteres junto con el *retriever* híbrido.

En el marco de la evaluación, la principal mejora propuesta es independizar el modelo evaluado del modelo juez. Utilizar para RAGAS un modelo distinto al que genera las respuestas reduce el sesgo de *self-preference* descrito en los apartados 4.1 y 6.2, y permite interpretar los valores absolutos de las métricas con mayor confianza.

El proyecto NATURCOR-IA, en el que se enmarca este trabajo, tiene como objetivo implementar un asistente virtual multilingüe (apartado 1.2), aspecto que

el sistema actual no aborda explícitamente. Llama 3.1 admite varios idiomas en la generación, pero el sistema no incorpora un modelo de *embeddings* con cobertura multilingüe unificada, lo que limita la representación coherente de consultas y fragmentos en distintos idiomas dentro de un mismo espacio vectorial. Modelos como LaBSE o multilingual-E5 están específicamente diseñados para esta tarea y resolverían también la representación coordinada de las versiones en inglés de las fichas, que actualmente se indexan como documentos independientes de sus equivalentes en español, mencionado en la sección 3.2. Por otro lado, el corpus utilizado en este trabajo es estático y se construyó a partir de una descarga única del portal. La descarga automatizada de nuevos documentos y la detección de cambios en los ya indexados, anticipadas en la sección 3.2, son necesarias para mantener el sistema actualizado y alineado con el contenido publicado en el portal y se proponen como línea de trabajo.

Por último, el sistema construido funciona como prototipo de consola, suficiente para el desarrollo y la evaluación experimental, pero no pensado para el uso final. Una interfaz web o la integración en un asistente conversacional ya existente, en línea con el resultado principal del proyecto NATURCOR-IA descrito en la sección 1.2, son los pasos siguientes para acercar el sistema al usuario final. Esta evolución haría posible una validación con usuarios reales que complemente la evaluación automática realizada en este trabajo y permita matizar la recomendación descrita en la sección 6.1 con criterios de calidad observados en condiciones de uso.

Referencias bibliográficas

1. Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahlooq, A., Bakhtiari, A., Behl, H., Bubeck, S., ... y Zhou, X. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv preprint arXiv:2404.14219*.
2. Asai, A., Wu, Z., Wang, Y., Sil, A., y Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511*.
3. BAAI. (2023). BGE embedding models. Beijing Academy of Artificial Intelligence. <https://huggingface.co/BAAI/bge-large-en>.
4. Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., y Abdelrazek, M. (2024). Seven Failure Points When Engineering a Retrieval Augmented Generation System. *arXiv preprint arXiv:2401.05856*.
5. Chroma. (2026). *Chroma documentation*. <https://docs.trychroma.com>.
6. Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2.^a ed.). Lawrence Erlbaum.
7. Confident AI. (2026). *DeepEval documentation* [Incluye métricas: Answer Relevancy, Faithfulness, Contextual Precision, Contextual Recall, Contextual Relevancy]. Recuperado de <https://docs.confident-ai.com/>.
8. Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1–30.
9. Devlin, J., Chang, M.-W., Lee, K., y Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.
10. Es, S., James, J., Espinosa-Anke, L., y Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*.

11. Feng, F., Yang, Y., Cer, D., Arivazhagan, N., y Wang, W. (2020). Language-agnostic BERT Sentence Embedding. *arXiv preprint arXiv:2007.01852*.
12. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., y Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.
13. Hinton, G., Vinyals, O., y Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.
14. Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
15. Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., y Liu, T. (2024). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 1(1), 1–58.
16. Irugalbandara, C., Mahendra, A., Daynauth, R., Kasthuri Arachchige, T., Dantanarayana, J., Flautner, K., Tang, L., Kang, Y., y Mars, J. (2024). Scaling Down to Scale Up: A Cost-Benefit Analysis of Replacing OpenAI's LLM with Open Source SLMs in Production. *arXiv preprint arXiv:2312.14972*.
17. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Chen, D., Dai, W., Chan, H. S., Madotto, A., y Fung, P. (2022). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
18. Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Le Scao, T., Lavril, T., Wang, T., Lacroix, T., y El Sayed, W. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.
19. Johnson, J., Douze, M., y Jégou, H. (2017). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
20. Junta de Andalucía. (2026). *Ventana del Visitante de los Espacios Naturales de Andalucía*. Consejería de Sostenibilidad, Medio Ambiente y Economía Azul. <https://www.juntadeandalucia.es/medioambiente/portal/web/ventana-delvisitante>.
21. Kamradt, G. (2023). *Semantic Chunking*. FullStackRetrieval. <https://github.com/FullStackRetrieval-com/RetrievalTutorials>.

22. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., y Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *arXiv preprint arXiv:2004.04906*.
23. LangChain. (2024a). CharacterTextSplitter. LangChain Reference. <https://reference.langchain.com/python/langchain-text-splitters/character/CharacterTextSplitter>.
24. LangChain. (2024b). *How to recursively split text by characters*. LangChain Documentation. https://python.langchain.com/docs/how_to/recursive_text_splitter/.
25. LangChain. (2024c). PyPDFLoader. LangChain Reference. https://reference.langchain.com/python/langchain-community/document_loaders/pdf/PyPDFLoader.
26. LangChain. (2024d). RecursiveCharacterTextSplitter. LangChain Reference. <https://reference.langchain.com/python/langchain-text-splitters/character/RecursiveCharacterTextSplitter>.
27. LangChain. (2024e). *Text splitters*. LangChain Documentation. https://python.langchain.com/docs/concepts/text_splitters/.
28. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
29. Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Proceedings of the ACL Workshop on Text Summarization Branches Out*, 74-81.
30. Ma, X., Gong, Y., He, P., Zhao, H., y Duan, N. (2023). Query Rewriting for Retrieval-Augmented Large Language Models. *arXiv preprint arXiv:2305.14283*.
31. Malkov, Y. A., y Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836.
32. Meta AI. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
33. Mikolov, T., Chen, K., Corrado, G., y Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.

34. Muennighoff, N., Tazi, N., Magne, L., y Reimers, N. (2022). MTEB: Massive Text Embedding Benchmark. *arXiv preprint arXiv:2210.07316*.
35. Nogueira, R., y Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
36. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., y Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.
37. Panickssery, A., Bowman, S. R., y Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations. *arXiv preprint arXiv:2404.13076*.
38. Papineni, K., Roukos, S., Ward, T., y Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311-318.
39. Pennington, J., Socher, R., y Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543.
40. Pinecone. (2026). *Pinecone documentation*. <https://docs.pinecone.io>.
41. Qdrant. (2026). *Qdrant documentation*. <https://qdrant.tech/documentation>.
42. Ragas. (2026). *Context Precision*. Ragas Documentation. Recuperado de https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/.
43. Reimers, N., y Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084*.
44. Robertson, S., y Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
45. Saad-Falcon, J., Khattab, O., Potts, C., y Zaharia, M. (2023). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *arXiv preprint arXiv:2311.09476*.

46. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., y Lample, G. (2023). *LLaMA: Open and Efficient Foundation Language Models*. <http://arxiv.org/abs/2302.13971>.
47. TruEra. (2026). *TruLens documentation*. Recuperado de <https://www.trulens.org/docs/>.
48. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
49. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., y Wei, F. (2022). Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533*.
50. Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., y Wei, F. (2024). Multilingual E5 Text Embeddings: A Technical Report. *arXiv preprint arXiv:2402.05672*.
51. Weaviate. (2026). *Weaviate Cloud documentation*. <https://docs.weaviate.io/cloud>.
52. Zhao, Y., Fan, Z., Cao, Y., Lyu, Z., y Li, J. (2026). RewriteGen: Autonomous Query Optimization for Retrieval-Augmented Large Language Models via Reinforcement Learning. *Electronics*.
53. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., y Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*

Anexo A. Repositorio de código y datos

El código del sistema y el material experimental están disponibles en el siguiente repositorio público:

<https://github.com/humainslab/2026-TFM-AnastasiaKuzmina>

El repositorio se organiza en las siguientes carpetas:

- **Raíz:** el código del bot conversacional en sus distintas versiones (`andalucia_bot.py`) y el script base de evaluación (`evaluate_ragas.py`), correspondientes al sistema descrito en los capítulos 3 y 4.
- **data/:** el corpus de enlaces a los PDFs y el conjunto de preguntas utilizado en la evaluación.
- **experimentos/:** una subcarpeta por cada experimento del capítulo 5, con los scripts de evaluación específicos de cada configuración y sus resultados en formato Excel. La subcarpeta *statistics* contiene el script de análisis estadístico que produce las tablas del capítulo 5.
- **outputs/:** los resultados de las distintas versiones del sistema.
- **Informe&Documentos/:** documentación asociada al TFM.

Anexo B. *Prompts* del sistema

Este anexo recoge los *prompts* utilizados en el sistema. El *prompt* del sistema y su evolución entre versiones se describen en la sección 3.4.2, y el *prompt* de contextualización del *retriever* en la sección 3.4.1. Los *prompts* específicos de las técnicas de RAG Avanzado evaluadas en el capítulo 5 se incluyen también en este anexo y se describen en las secciones correspondientes.

B.1. *Prompt* del sistema, versión inicial

Versión general utilizada sin cambios en las implementaciones v1, v2 y v2.1 del bot.

```
Eres un asistente experto en turismo y medio ambiente de Andalucía.  
Responde a las preguntas basandote unicamente en el contexto  
proporcionado a continuacion.  
Si no conoces la respuesta, indicalo claramente.  
Responde siempre en español.  
  
{context}
```

B.2. *Prompt* del sistema, versión v2.2

Versión estructurada en cinco reglas introducida en la implementación v2.2 del bot.

```
Eres un guia experto en turismo activo y naturaleza de Andalucía.  
Tu objetivo es asesorar al usuario de forma fiable y profesional.  
  
REGLA 1 - FUENTES DE INFORMACION:  
Usa EXCLUSIVAMENTE el contexto tecnico proporcionado para responder  
sobre:  
- Longitud, desnivel, duracion y dificultad de senderos concretos
```

- Flora y fauna de parques o rutas específicas
- Puntos de inicio, aparcamiento, señalización y balizamiento
- Restricciones, permisos o normativa de espacios naturales

Si esta información no está en el contexto, indicarlo claramente:

"No tengo datos técnicos sobre esta ruta en concreto, pero puedo orientarte sobre opciones similares."

REGLA 2 - CONOCIMIENTO GENERAL PERMITIDO:

Puedes usar tu conocimiento general únicamente para:

- Geografía: provincias de Andalucía, ubicación de municipios, distancias entre localidades
- Clima y estacionalidad general de la región
- Recomendaciones genéricas de equipamiento (agua, calzado, ropa)
- Clasificación estándar de dificultad
(que significa fácil, medio, difícil en senderismo)

REGLA 3 - FILTRO GEOGRÁFICO:

Si los datos técnicos disponibles corresponden a una zona distinta a la consultada, indicarlo antes de dar la información:

"Los datos que tengo son de [zona], que está a aproximadamente [distancia] de donde mencionas. Te los comparto por si te son útiles."

REGLA 4 - DOCUMENTOS Y ARCHIVOS GPX:

Si el usuario solicita mapas, archivos GPX, PDF de rutas u otros documentos descargables, explica que no puedes enviar archivos directamente y dirígelo al portal oficial:

"Para descargar el track GPX o el PDF oficial de esta ruta, puedes encontrarlo en el portal de la Junta de Andalucía: juntadeandalucia.es/medioambiente -- sección Red de Senderos de Andalucía."

REGLA 5 - FORMATO:

Usa ****negrita**** para nombres de rutas y conceptos clave.

Usa listas con guiones para que la información sea escaneable.

Se conciso: no repitas información ya mencionada en el historial del chat.

Información técnica disponible:

{context}

B.3. *Prompt* de contextualización del *retriever*

Prompt utilizado por el *history-aware retriever* para reformular la pregunta del usuario en función del historial de la conversación.

```
Dada la historia del chat y la ultima pregunta del usuario,  
que podria hacer referencia al contexto en la historia del chat,  
formula una pregunta independiente que pueda entenderse sin la  
historia del chat. NO respondas a la pregunta, solo reformulala  
si es necesario y devuélvela.
```

B.4. *Prompt* de *Multi-Query*

Prompt utilizado por *MultiQueryRetriever* para generar las subconsultas alternativas a partir de la pregunta del usuario. Las variantes MQ-B3x4 y MQ-B4x3 comparten este *prompt* con la diferencia en el número de subconsultas y en el valor de k de cada subconsulta.

```
Eres un asistente experto en turismo activo y senderismo en Andalucía.  
Tu tarea es generar 3/4 versiones diferentes de la pregunta del usuario  
para recuperar documentos relevantes de una base de datos vectorial  
sobre  
rutas de senderismo y espacios naturales de Andalucía.  
Al generar multiples perspectivas, ayudas a superar las limitaciones  
de la búsqueda por similitud semantica.  
Proporciona EXACTAMENTE 3/4 preguntas alternativas, una por linea,  
sin numeracion, sin explicaciones, sin texto adicional.  
  
Pregunta original: {question}
```

B.5. *Rewrite-Retrieve-Read, AD*

Prompt utilizado en las configuraciones RRR-AD y RRR-AD-300 para reformular la pregunta adaptándola al vocabulario del corpus.

Mensaje *system*:

```
Eres un asistente que adapta preguntas al lenguaje tecnico de documentos de senderismo andaluz. Dado el contexto recuperado y la pregunta original, reformula la pregunta usando los terminos y estilo del contexto para mejorar la busqueda. Devuelve UNICAMENTE la pregunta reformulada, sin explicaciones ni introduccion.
```

Mensaje *human*:

```
Pregunta original: {question}
```

```
Contexto recuperado:  
{context}
```

```
Pregunta reformulada:
```

B.6. *Rewrite-Retrieve-Read, GE*

Prompt utilizado en la configuración RRR-GE para reformular la pregunta, haciéndola más específica geográficamente y por parámetros del recorrido.

Mensaje *system*:

```
Eres un asistente que precisa preguntas para mejorar la busqueda en una base de datos de senderismo andaluz. Dada la pregunta original y el contexto recuperado, reescribe la pregunta haciendola mas concreta: identifica la zona geografica (provincia, parque natural, sierra), los parametros mencionados (dificultad, duracion, longitud) y los explicita en la pregunta reformulada. Si la pregunta es general, manten la generalidad pero usa nombres propios cuando el contexto los sugiera. Devuelve UNICAMENTE la pregunta reformulada.
```

Mensaje *human*:

Pregunta original: {question}

Contexto recuperado:
{context}

Pregunta reformulada: